VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



PARALLEL COMPUTING

Assignment 1

Mutex Lock, Sense-Reversing Barrier và Concurrent Counter

Cài đặt, kiểm tra tính đúng đắn và nghiên cứu hiệu năng

Advisor: Assoc. Prof. Trần Văn Hoài
PhD. Diệp Thanh Đăng
Students: Họ và tên thành viên 1 - MSHV.
Họ và tên thành viên 2 - MSHV.
Họ và tên thành viên 3 - MSHV.

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Giới thiệu và câu hỏi nghiên cứu **2**

2 Nền tảng thí nghiệm **3**

3 Cài đặt **4**
3.1 Mutex lock 4
3.2 Sense-reversing barrier 5
3.3 Concurrent counter 6

4 Kiểm tra tính đúng đắn **7**
4.1 Chiến lược 7
4.2 Mutex lock 7
4.3 Sense-reversing barrier 8
4.4 Concurrent counter 8
4.5 Mutation testing 9
4.6 Giới hạn của phương pháp 10

5 Phương pháp đo **10**

6 Kết quả và phân tích **12**
6.1 Mutex lock 12
6.2 Barrier 18
6.3 Concurrent counter 20
6.4 So sánh giữa các máy 23

7 Các yếu tố đe dọa độ tin cậy **27**

8 Kết luận **28**

References **30**

A Cách build và chạy lại **30**

B Tên các cài đặt **31**

C Kết quả kiểm tra, đợt đo và hình bổ sung **32**

D Bảng số liệu của hai máy còn lại **34**

E Cấu trúc mã nguồn **37**

F Khai báo sử dụng công cụ AI **37**

1 Giới thiệu và câu hỏi nghiên cứu

Nhóm cài đặt bằng C++ mutex lock (TAS, TTAS, MCS), sense-reversing barrier và concurrent counter (lock, CAS, fetch-and-add). Nhóm kiểm tra tính đúng đắn của các cài đặt và đo khả năng mở rộng trên máy shared-memory, so với cấu trúc của thư viện. Phần 4 của đề (MPI-3 RMA [6]) không bắt buộc và nhóm không làm.

- *Cache line.* Đơn vị (128 byte trên hai máy Apple, 64 byte trên `LAP1184`) mà phần cứng giữ nhất quán giữa các core. Nhiều core cùng đọc được, nhưng ghi thì một core phải giữ line độc quyền và làm mất hiệu lực bản sao ở core khác.
- *Tranh chấp.* Nhiều thread cùng muốn vào critical section. `OUTSIDE_WORK` $= 0$ là tranh chấp tối đa.
- *Vòng lặp đo.* Mỗi vòng, thread lấy lock, gọi `is_prime(N)` trong critical section, nhả lock, rồi gọi `is_prime(N)` thêm OW lần ở ngoài (OW viết tắt `OUTSIDE_WORK`). p là số thread. Mục Phương pháp đo định nghĩa đầy đủ các ký hiệu, độ chậm và độ công bằng.
- *TAS, TTAS, MCS.* TAS lặp `exchange` lên cờ lock. TTAS đọc cờ tới khi rảnh rồi mới `exchange` [1]. MCS xếp hàng, mỗi thread chờ trên cờ riêng [5].
- *Episode.* Một lần cả p thread cùng vượt barrier [2].
- *FAA, CAS.* Fetch-and-add cộng và trả giá trị cũ trong một bước. Compare-and-swap chỉ ghi nếu giá trị chưa đổi, nếu không thì thất bại và phải thử lại.

Mục Kết quả giải thích số đo chủ yếu bằng số lần một cache line đi giữa các core trên đường tới hạn [3], cùng với scheduler và hai loại core của máy. Các hình đánh dấu hai mốc trên trục số thread, là số P-core và số core của máy (trên `apple-m5` là 4 và 10). P-core là core hiệu năng cao. E-core là core tiết kiệm điện, chậm hơn, và hai loại nằm ở hai cụm có cache L2 riêng.

RQ1 (đúng đắn). Các cài đặt có đúng không, và bộ kiểm tra có đủ nhạy để bắt lỗi cài đặt?

RQ2 (lock: mở rộng). TAS, TTAS, MCS mở rộng thế nào theo số thread, `OUTSIDE_WORK` và N, so với lock thư viện?

RQ3 (lock: backoff và công bằng). Backoff giúp hay hại, và ở chế độ nào? Lock nhanh nhất có công bằng không?

RQ4 (barrier). Chi phí một episode tăng thế nào theo số thread, theo cách chờ, so với barrier thư viện?

RQ5 (counter). Thứ tự hiệu năng giữa FAA, CAS (có/không backoff) và counter dùng lock, và vì sao?

RQ6 (giữa các máy). Các kết luận trên có giữ nguyên trên máy khác không?

RQ	Thí nghiệm	Hình, bảng
RQ1	stress, TSan, mutation	Bảng 3, 9 (phụ lục)
RQ2	lock × thread × OW × N	Hình 1, 10 (phụ lục); Bảng 4, 5
RQ3	sáu biến thể backoff, thời gian ngủ thật	Hình 2, 3
RQ4	episode/giây, bảy barrier	Hình 4; Bảng 6
RQ5	thời gian, số CAS hỏng	Hình 5, 6; Bảng 7
RQ6	cùng quy trình trên `C4M7Y670NT` và `LAP1184`	Hình 7, 8, 9; Bảng 2 và các bảng ở phụ lục

Bảng 1: Câu hỏi nghiên cứu, thí nghiệm và hình, bảng trả lời.

Đóng góp của báo cáo:

- cài đặt ba nhóm cấu trúc kèm lập luận memory order cho từng thao tác atomic;

- bộ kiểm tra (stress, watchdog, ThreadSanitizer, schedule fuzzing) được đánh giá bằng mutation testing;
- quy trình đo một lệnh cho đợt đo chính, sinh lại mọi hình và bảng của đợt đo đó, và thêm máy chỉ cần thêm tên máy vào danh sách của báo cáo;
- so sánh số đo với cận dưới lý thuyết.

Mục Nền tảng thí nghiệm mô tả máy đo, và mục Cài đặt trình bày thuật toán. Mục Kiểm tra tính đúng đắn trả lời RQ1. Mục Phương pháp đo mô tả cách đo, và mục Kết quả và phân tích trả lời RQ2–RQ6. Báo cáo kết thúc bằng mục Các yếu tố đe dọa độ tin cậy, mục Kết luận và phụ lục.

2 Nền tảng thí nghiệm

Đề yêu cầu số nền tảng shared-memory không ít hơn số thành viên. Nhóm có ba thành viên và ba máy: `apple-m5` (Apple M5, macOS), `C4M7Y670NT` (Apple M3 Pro, macOS) và `LAP1184` (Intel x86-64, Linux trong máy ảo). Bảng 2 tóm tắt cấu hình. Mục Kết quả và phân tích dựa trên `apple-m5`. RQ6 so ba máy.

Máy `apple-m5` là một Apple M5 với 10 core không đồng nhất, gồm 4 P-core (performance) và 6 E-core (efficiency). Mỗi cụm có cache L2 riêng, 16 MiB cho cụm P-core và 6 MiB cho cụm E-core. Máy không có SMT (simultaneous multithreading, nhiều luồng phần cứng chạy chung một core).

Thứ nhất, khi số thread vượt 4, một phần thread chạy trên E-core chậm hơn, và cache line dùng chung phải đi qua lại giữa hai cụm. Thứ hai, loại core của mỗi thread do scheduler của macOS quyết định và có thể đổi giữa các lần chạy. Thứ ba, AArch64 có memory model yếu hơn x86-TSO. Các memory order khác nhau sinh ra các lệnh máy khác nhau (store `relaxed` là `str`, store `release` là `stlr`). Vì vậy lỗi chọn sai memory order có thể quan sát được trên máy này, trong khi trên x86 nó thường không lộ ra. CPU có các lệnh atomic LSE (Large System Extensions) của ARMv8.1, nên `fetch_add` và `compare_exchange` là một lệnh máy (`ldadd`, `cas`) chứ không phải vòng LL/SC.

Hệ điều hành báo cache line dài 128 byte. Mã nguồn đệm mọi biến dùng chung theo hằng `pc::kCacheLine`, bằng 128 trên Apple arm64 và 64 trên nền tảng khác. Nhóm kiểm tra rằng kích thước và căn chỉnh của mọi kiểu lock và của nút hàng đợi MCS đều bằng 128 byte trên máy này.

Phần mềm gồm macOS 27.0.1 và Apple clang 21.0.0 với cờ `-std=c++20 -O2`. Apple clang không nhận `-fopenmp`, và macOS không có `pthread_spinlock_t` hay `pthread_barrier_t`. Vì vậy các baseline trên máy này là `std::mutex`, `pthread_mutex_t`, `os_unfair_lock`, `std::barrier` và barrier dựng từ mutex và condition variable.

Số đo chính trên `apple-m5` đến từ một đợt đo dài 1 giờ 26 phút, gồm 12 152 lần chạy hoàn thành (cùng 9 lần vượt thời hạn). Máy cắm sạc và chế độ tiết kiệm điện tắt. Load average một phút lúc bắt đầu là 6,73. Runner đã chờ tới khi load average xuống dưới ngưỡng 7,0 mới bắt đầu. Hai thí nghiệm phụ của RQ2 (khoảng trống khi trao lock, đo lại khi tải thấp) chạy riêng sau đó, trong 47 phút.

Máy `C4M7Y670NT` là một Apple M3 Pro với 12 core, gồm 6 P-core và 6 E-core, không có SMT. Mỗi cụm có cache L2 riêng, 16 MiB cho cụm P-core và 4 MiB cho cụm E-core. Máy chạy macOS 26.7.1 và Apple clang 21.0.0 với cùng cờ biên dịch và cùng các baseline như `apple-m5`. Hai máy Apple cùng kiến trúc, hệ điều hành và compiler. Chúng khác nhau ở số P-core và số core (`C4M7Y670NT` nhiều hơn 2 P-core và 2 core), nhưng cũng ở thế hệ chip (M3 Pro, M5), cache L2 và phiên bản macOS. Lưới thread thêm 24. Đợt đo dài 1 giờ 53 phút, gồm 13 244 lần chạy hoàn thành, đều qua phép kiểm kết quả (cùng 10 lần vượt thời hạn, đều là MCS quá tải). Máy cắm sạc, chế độ tiết kiệm điện tắt. Load average một phút lúc bắt đầu là 6,45.

Máy `LAP1184` là một laptop Intel Core i5-1335U. Theo thông số của Intel, CPU này có 2 P-core với Hyper-Threading và 8 E-core, tức 10 core và 12 luồng phần cứng. Cache L3 12 MiB dùng

chung. Nhóm đo trong Ubuntu 20.04 chạy trên WSL2, một máy ảo Hyper-V của Windows 11.

Thứ nhất, máy ảo có 12 CPU ảo và báo chúng là 6 core, mỗi core 2 luồng. Cách báo này không khớp với CPU thật. Thứ hai, mỗi CPU ảo là một thread do scheduler của Windows quyết định. x86-64 có memory model TSO, cache line 64 byte, và mã nguồn đệm theo 64 byte.

Phần mềm gồm g++ 11.5 với `-std=c++20 -O2 -fopenmp`. g++ 9 mặc định của Ubuntu 20.04 không có `<barrier>`, nên nhóm cài g++ 11 từ PPA `ubuntu-toolchain-r/test`. Ngoài các baseline chung, máy này có `pthread_spinlock_t`, `pthread_barrier_t`, `omp_lock_t` và `#pragma omp barrier`, nên có 13 lock, 9 barrier và 20 counter. Đợt đo dài 1 giờ 20 phút 36 giây, gồm 14 322 lần chạy hoàn thành (cùng 12 lần vượt thời hạn, Bảng 19). Load average một phút lúc bắt đầu là 6,06. `machine.txt`, sinh lại 3 phút sau đợt đo, ghi nguồn AC.

Bảng 2: Cấu hình các máy dùng để đo. Thông tin do `scripts/machine_info.sh` ghi lại lúc chạy đợt đo; kích thước cache line do hệ điều hành báo.

	`apple-m5`	`C4M7Y670NT`	`LAP1184`
CPU	Apple M5	Apple M3 Pro	13th Gen Intel(R) Core(TM) i5-1335U
Kiến trúc	arm64	arm64	x86_64
Số core	10 (4 P-core + 6 E-core)	12 (6 P-core + 6 E-core)	12 luồng phần cứng trên 6 core vật lý (máy ảo báo)
Cache line (hệ thống báo)	128 B	128 B	64 B
Cache line dùng để đệm trong mã	128 B	128 B	64 B
Cache L1d	128 KiB (P) / 64 KiB (E)	128 KiB (P) / 64 KiB (E)	48 KiB
Cache L2	16 MiB cho 4 P-core / 6 MiB cho 6 E-core	16 MiB cho 6 P-core / 4 MiB cho 6 E-core	1,25 MiB
Cache L3	–	–	12 MiB
RAM	16 GiB	36 GiB	7,6 GiB
Hệ điều hành	macOS 27.0.1 (26A434)	macOS 26.7.1 (25G241)	Ubuntu 20.04 LTS
Nhân	Darwin 27.0.0	Darwin 25.6.0	Linux 5.15.153.1-microsoft-standard-WSL2
Compiler	Apple clang version 21.0.0 (clang-2100.1.1.101)	Apple clang version 21.0.0 (clang-2100.3.34.2)	c++ (Ubuntu 11.5.0-10ubuntu1~20~ppa3) 11.5.0
Cờ biên dịch	`-std=c++20 -O2 -Wall -Wextra -pthread`	`-std=c++20 -O2 -Wall -Wextra -pthread`	`-std=c++20 -O2 -Wall -Wextra -pthread -fopenmp`
OpenMP	không (không có baseline OpenMP)	không (không có baseline OpenMP)	có
Lệnh chờ trong vòng spin	`isb`	`isb`	`pause`
Ghim thread vào core	không có (macOS trên Apple Silicon không có API ghim thread)	không có (macOS trên Apple Silicon không có API ghim thread)	có (`sched_setaffinity`) nhưng các benchmark không dùng
Nguồn điện lúc đo	cắm sạc	cắm sạc	cắm sạc (ghi sau đợt đo)
Thành viên chạy đo	*[điền]*	*[điền]*	*[điền]*

3 Cài đặt

3.1 Mutex lock

Mọi lock trong `include/lock.h` có đúng `acquire()` và `release()` không tham số như khung của đề. Với khai báo `using pc::lock_mcs;`, dòng `lock_mcs lock;` của khung đề biên dịch nguyên văn. Lấy lock dùng acquire, nhả lock dùng release. Cặp này làm mọi phép ghi trong critical section trước nhìn thấy được ở chủ lock sau. Trạng thái lock được khai báo `alignas(kCacheLine)`, nên mỗi lock chiếm riêng một cache line (128 byte trên hai máy Apple, 64 byte trên `LAP1184`). Nhờ vậy không có false sharing, tức hai biến độc lập nằm chung một line và làm mất hiệu lực line của

nhau.

TAS lặp `exchange(true, acquire)` (lệnh `swpab`) tới khi đọc được `false`. TAS nhả lock bằng `store(false, release)` (`stlrb`). TTAS chờ bằng `load(relaxed)` (`ldrb`) rồi mới `exchange`. Phép đọc chỉ là gợi ý, `exchange` thắng mới là phép acquire.

Mọi vòng spin gọi `cpu_relax()` giữa hai lần thử, tức là `pause` trên x86 và `isb` trên AArch64. Lệnh `isb` được chọn vì `yield` của Apple gần như không làm gì (0,3 ns so với 12,1 ns của `isb`, cùng phép đo). `isb` dừng pipeline trong khoảng thời gian đó, nên thread chờ đọc lock thưa hơn.

MCS [5] giữ `tail_` (`nullptr` là rảnh). Mỗi thread chờ trên cờ `locked` trong nút của mình, nên mỗi lần chuyển giao chỉ ghi một line của một thread, theo thứ tự FIFO. Trong các đoạn mã dưới đây, `relaxed`, `acquire`, `release`, `acq_rel` viết tắt cho `std::memory_order_*`.

```
me->next.store(nullptr, relaxed); me->locked.store(true, relaxed);  // not yet reachable
pred = tail_.exchange(me, acq_rel);                // release: publish node; acquire: take lock
if (pred == nullptr) return;
pred->next.store(me, release);
while (me->locked.load(acquire)) cpu_relax();      // spin on own cache line
// release(): succ = next.load(acquire); if null, CAS tail_ me -> null (release);
//            on failure wait until next != null (acquire)
succ->locked.store(false, release);                // hand over
```

Hai phép ghi relaxed an toàn vì nút chỉ trở nên với tới được qua hai phép release theo sau. Nếu CAS `tail_` thất bại, một thread đã đổi `tail_` nhưng chưa nối vào `next`. Chủ lock phải chờ nó, vì nếu bỏ đi thì thread đó chờ mãi. Vòng chờ này ngắn, vì thread kia đã `exchange` xong và lệnh kế tiếp của nó là ghi `next`. Một thread chỉ vào critical section khi nó thấy `tail_` rỗng hoặc khi thread trước nó đặt `locked` của nó về `false`, nên tại mỗi lúc chỉ có một chủ lock. Hàng đợi theo thứ tự FIFO, nên không thread nào bị đói. Trên AArch64 các thao tác thành `swpal` (`exchange` lên `tail_`), `stlr` (nối `next`), `ldaprb` (chờ `locked`), `ldapr` (`release()` đọc `next`), `casl` (CAS `tail_`) và `stlrb` (trao lock). Nút `mcs_node` cũng chiếm riêng một line.

Giới hạn thứ nhất của MCS là một thread không giữ được hai lock MCS cùng lúc, vì nút là biến `thread_local` dùng chung cho mọi `lock_mcs` do giao diện không có tham số. Giới hạn thứ hai là lock được giao cho đúng một thread, nên nếu thread đó bị preempt thì mọi thread phía sau phải chờ scheduler.

`lock_tas_bo<B>` và `lock_ttas_bo<B>` gọi chính sách `B` sau mỗi lần thử thất bại. TTAS chỉ lùi lại khi thua `exchange` (BackoffLock [3]). Memory order giống TAS và TTAS, nên lập luận về tính đúng ở trên giữ nguyên. `B` là một trong ba chính sách. `sleep` theo mẫu của đề, gọi `sleep_for` với `delay` nhân đôi từ 100 ns tới `MAX_DELAY` = 100 000 ns. `spin` quay ngẫu nhiên, với cửa sổ nhân đôi từ 4 tới 1024 lần `cpu_relax`. `yield` quay 64 lần rồi gọi `yield`. `sleep_for` là system call. Trên `apple-m5`, yêu cầu 100 ns ngủ trung vị 2 670 ns (nhóm đo bằng `bench/sleep_latency.cpp`).

Lock thư viện được bọc inline sau cùng giao diện. Đó là `std::mutex`, `pthread_mutex_t`, `os_unfair_lock` (macOS), `pthread_spinlock_t` (Linux), `omp_lock_t` (khi có OpenMP). Bảng 8 (phụ lục) liệt kê tên mọi cài đặt và máy có chúng.

3.2 Sense-reversing barrier

Barrier cho n thread bảo đảm không thread nào ra khỏi lần gọi `wait()` thứ k trước khi cả n thread đã gọi lần thứ k. Barrier cũng dùng lại được qua mọi episode. Barrier một cờ "mở/đóng" không dùng lại được. Đóng cờ sớm thì thread chậm chờ mãi, đóng muộn thì thread nhanh qua luôn episode sau.

Sense-reversing barrier [2] không đặt lại cờ mà đảo nó mỗi episode. Mỗi thread chờ cờ bằng *local sense* của mình, nên hai episode kề nhau chờ hai giá trị khác nhau. Giữa hai episode, mọi local sense bằng `!sense_`. Cờ không thể đảo lại trước khi thread đang chờ rời đi, vì lần đảo sau cần đủ n lần tới, trong đó có lần tới của chính thread này.

```
const bool my_sense = local_[tid].sense;
```

```
if (count_.fetch_add(1, acq_rel) == n_ - 1) {      // last arriver
    count_.store(0, relaxed);                      // 1. reset for the next episode
    sense_.store(my_sense, release);               // 2. THEN release the others
} else {
    while (sense_.load(acquire) != my_sense) backoff.pause();
}
local_[tid].sense = !my_sense;
```

Thread tới cuối phải đặt lại bộ đếm trước khi đảo cờ. Nếu đảo trước, thread vừa được thả có thể tới episode sau, rồi bị phép đặt lại muộn xoá mất lần tới, và mọi thread chờ mãi. Bộ kiểm tra bắt được biến thể đảo thứ tự này (Bảng 3).

`fetch_add` cần acq_rel (`ldaddal`). Nửa release công bố phép ghi trước barrier. Các `fetch_add` lên `count_` nối nhau thành một release sequence, nên nửa acquire của thread tới cuối đồng bộ với mọi thread tới trước. Phép ghi release lên `sense_` (`stlrb`) chuyển tiếp các phép ghi đó cho mọi thread đang chờ, vì chúng đọc cờ bằng acquire (`ldaprb`). Với chỉ release (`ldaddl`), bài stress bắt được một thread qua barrier rồi đọc ô cũ của thread khác (Bảng 3). Phép đặt lại relaxed đủ vì nó đứng trước phép ghi release lên `sense_`.

`count_`, `sense_` và mỗi local sense nằm trên các cache line riêng, để lần tới không làm mất hiệu lực line đang được chờ. Local sense nằm trong barrier, đánh chỉ số bằng `tid`, vì một biến `thread_local` sẽ bị hai barrier dùng xen kẽ làm hỏng.

Các cách chờ gồm quay (`sr`), ba chính sách backoff như lock, và chờ chặn (`sr_block`) bằng `std::atomic::wait` và `notify_all`. `sr_block` giữ nguyên giao thức và memory order ở trên. `wait(old)` kiểm lại giá trị một cách atomic trước khi ngủ, nên không mất lần đánh thức nào. Theo mã nguồn libc++ và libstdc++, `wait` quay một lúc rồi mới chặn thật, nên `sr_block` thực chất là quay ngắn rồi ngủ. Các barrier thư viện gồm `std::barrier`, barrier dựng từ mutex và condition variable (`condvar`), `pthread_barrier_t` (Linux) và `omp barrier`. Các giới hạn là số thread cố định và barrier tập trung tốn $O(n)$ trên đường tới hạn. Biến thể quay vẫn đúng khi số thread vượt số core, nhưng mỗi episode khi đó có thể tốn cả một lượt lập lịch.

3.3 Concurrent counter

Counter bắt đầu từ 0. `increment()` cộng 1 và trả về giá trị *trước* khi tăng. M lần tăng phải trả về đúng $0, \ldots, M-1$, mỗi giá trị một lần, và theo thứ tự thời gian thực (linearizable [4]). Ba cài đặt là `counter_lock<L>` (biến `long` trong critical section của mỗi lock ở phần 1), `counter_cas` (lock-free, có biến thể backoff) và `counter_faa` (wait-free khi phần cứng có lệnh cộng atomic, tức LSE trên ARM hoặc `lock xadd` trên x86). Trên `apple-m5` có 19 counter.

```
long old = value_.load(relaxed);                   // a guess; the CAS checks it
while (!value_.compare_exchange_weak(old, old + 1, Order, relaxed)) {
    ++failed_;                                     // thread_local, failure path only
    backoff.pause();
}
return old;
// fetch-and-add: return value_.fetch_add(1, Order);
```

Thứ tự mặc định là `relaxed`, vì tính atomic của phép đọc-sửa-ghi và thứ tự toàn phần của mọi phép ghi lên một biến atomic đúng với mọi memory order. Điểm tuyến tính hoá là phép đọc-sửa-ghi (`fetch_add` hoặc CAS thành công). Một phép đọc-sửa-ghi luôn đọc giá trị mới nhất của biến trong thứ tự sửa đổi, nên không hai lần tăng đọc cùng một giá trị. Counter cũng không bảo vệ dữ liệu nào khác. Biến thể `faa_seqcst`, `cas_seqcst` đo cái giá của `seq_cst`. Counter dùng lock không có atomic riêng. Nó đúng nhờ cặp release–acquire của lock.

Apple clang bật LSE mặc định. `fetch_add` là một lệnh `ldadd` (`ldaddal` với `seq_cst`), không vòng lặp. CAS là một lệnh `cas` (`casal`), chỉ hỏng khi giá trị thật sự đã đổi, nên mỗi lần `cas_fail` là một xung đột thật. Khi tắt LSE, cả hai thành `ldxr`/`stxr` (LL/SC), và CAS weak có thể hỏng giả. Trên x86-64 hai thứ tự sinh cùng một lệnh (`lock xadd`, `lock cmpxchg`).

Giá trị atomic nằm riêng một cache line. Trong `counter_lock`, lock và giá trị nằm trên hai line để thread chờ không làm mất hiệu lực line của giá trị. Vì vậy mỗi lần tăng ghi lên hai line. Số lần CAS hỏng được đếm trong biến `thread_local` để không thêm một line bị tranh chấp. Về giới hạn, counter CAS chỉ lock-free nên một thread có thể bị đói. `counter_lock` mang theo giới hạn của lock bên trong.

4 Kiểm tra tính đúng đắn

4.1 Chiến lược

Nhóm kiểm hai nhóm tính chất. Tính an toàn (safety) nghĩa là không có hai thread cùng ở trong critical section, không thread nào ra khỏi barrier sớm, không giá trị counter nào bị trả về hai lần. Tính sống còn (liveness) nghĩa là chương trình không treo. Công bằng chỉ được đo.

Cùng một bài kiểm tra dạng template chạy cho mọi cài đặt, kể cả lock và barrier của thư viện.

Nhóm dùng bốn kỹ thuật:

- stress test với ít thread (1–4), với số luồng phần cứng và với gấp đôi số đó (`apple-m5`: 10 và 20; `C4M7Y670NT` và `LAP1184`: 12 và 24); ở mức gấp đôi, thread bị tước core giữa giao thức;
- watchdog, thread giám sát báo `FAIL` khi bài kiểm tra ngừng tiến triển;
- ThreadSanitizer (TSan), công cụ của compiler báo hai truy cập xung đột, ít nhất một không atomic, mà không có quan hệ happens-before (atomic riêng của bài kiểm tra dùng `relaxed` để không tự tạo thứ tự, trừ đồng hồ `seq_cst` của lượt kiểm thời gian thực);
- mutation testing, chạy bộ kiểm tra trên các biến thể cố ý làm hỏng.

Barrier dùng thêm schedule fuzzing. Counter dùng thêm phép kiểm thứ tự thời gian thực.

Bảng 9 (phụ lục) tóm tắt kết quả trên ba máy. Mọi cài đặt đạt mọi bài kiểm tra, TSan không báo data race nào trên mã đúng, và bộ kiểm tra bắt cả 14 biến thể hỏng. Bản TSan chạy với kích thước nhỏ hơn (ít vòng hơn 10–20 lần) vì TSan làm chương trình chậm đi nhiều. Đầu ra nằm trong `results/<máy>/`.

4.2 Mutex lock

Trong critical section, bài kiểm tra tăng một biến đếm không atomic (đọc, cộng một, ghi) và đếm số thread đang ở bên trong. Vi phạm hiện ra trực tiếp, hoặc qua một lần cập nhật bị mất. Lượt đầu chạy số vòng cố định: 1 thread lấy lock 200 000 lần, 2 thread mỗi thread 100 000 lần, và số luồng phần cứng mỗi thread 20 000 lần. Lượt sau chạy trong 300 ms với 4 thread, số luồng phần cứng và gấp đôi số đó (`apple-m5`: 4, 10, 20; hai máy còn lại: 4, 12, 24). Lượt gấp đôi thử lock khi quá tải. Watchdog kết thúc một bài kiểm tra con sau 20 s và báo deadlock.

Từ số lần lấy lock của từng thread, bài kiểm tra in tỉ số số lần lấy lock lớn nhất / nhỏ nhất (1,00 là chia đều). Khi mỗi thread có một core, `mcs` chia gần như đều (1,00 ở 4 thread, 1,01 ở 10 thread), đúng tính chất FIFO của hàng đợi MCS [5]. Ở 20 thread, MCS chỉ lấy lock 15 221 lần trong 300 ms (so với 2 705 564 lần ở 4 thread), với tỉ số 311,40. Số đo này phù hợp với việc lock được trao cho một thread đang không có core (cơ chế bàn ở RQ2).

Trên `C4M7Y670NT` (12 core), MCS vẫn chia đều ở 12 thread (tỉ số 1,00, 1 274 328 lần lấy lock), nhưng ở 24 thread chỉ lấy lock 10 657 lần, với tỉ số 222,25. Trên cả hai máy Apple, MCS chỉ sụp khi số thread vượt số core.

Trên `LAP1184`, MCS sụp sớm hơn. Tỉ số là 1,00 ở 4 thread, nhưng MCS đã sụp ở 12 thread, bằng số CPU ảo, với 10 186 lần lấy lock (so với 3 275 092 ở 4 thread). Ở 24 thread chỉ còn 1 628 lần. Ở 12 thread, nhóm cho rằng một CPU ảo đã bị Windows lấy core trong lúc giữ lượt của hàng đợi.

Ở bản TSan trên `LAP1184`, các lượt 100 ms với 24 thread không lấy được lock lần nào (`mcs` và

`ttas_bo_spin` ở 12 thread cũng 0). Vì vậy trên máy này TSan chỉ thực sự kiểm các lượt số vòng cố định và các lượt 4–12 thread.

Bản TSan trên `LAP1184` bỏ qua `omp_lock` và in ra việc bỏ qua. Khi không bỏ qua, TSan báo một data race trên biến đếm mà `omp_lock` bảo vệ. Nhóm cho rằng đây là cảnh báo giả, vì `omp_set_lock` và `omp_unset_lock` nằm trong libgomp, thư viện không được instrument. Vì vậy TSan không thấy thứ tự mà lock tạo ra. Cùng lock đó đạt stress test ở bản thường.

4.3 Sense-reversing barrier

Ở episode e (lần thứ e cả nhóm đi qua barrier), mỗi thread ghi e vào ô của mình, gọi `wait`, rồi đọc mọi ô. Một ô nhỏ hơn e nghĩa là thread này qua sớm. Một ô lớn hơn $e + 1$ nghĩa là thread khác đã qua episode kế tiếp mà không chờ. Các ô là atomic `relaxed`, nên điều kiện chỉ đúng nhờ thứ tự do barrier tạo ra.

Phép kiểm handoff dùng xen kẽ hai barrier với các ô là biến thường. Ở mọi bản, thread phải đọc đúng số episode từ mọi ô. Phép kiểm này bắt lỗi khi một thread dùng hai barrier. Dưới TSan, nó kiểm thêm memory order của barrier.

Mỗi barrier chạy sáu pha: 1 thread; 4 thread dùng lại barrier qua tới 300 000 episode liên tiếp; số luồng phần cứng; 4 thread có độ trễ ngẫu nhiên; gấp đôi số luồng phần cứng; và phép kiểm handoff với 4 thread (`apple-m5`: 1, 4, 10, 20; hai máy còn lại: 1, 4, 12, 24). Barrier chậm có số episode mục tiêu nhỏ hơn. Mỗi pha có ngân sách 1,5 s. Watchdog báo treo khi không thread nào tiến trong 10 s.

Bản schedule fuzzing chèn một độ trễ ngẫu nhiên vào mỗi điểm giữa hai bước của giao thức, để một thread bị chậm đúng giữa hai bước.

Trên `apple-m5`, pha 20 thread của `sr` và `sr_bo_spin` hết ngân sách sau 64 và 80 episode. Trên `C4M7Y670NT`, pha 24 thread của hai barrier này hết ngân sách sau 104 và 97 episode. Trên cả hai máy Apple, bản TSan bỏ phép kiểm handoff riêng cho `std_barrier`. Lý do là libc++ của macOS không được instrument, nên TSan báo cảnh báo giả.

Trên `LAP1184`, nhiều pha hết ngân sách 1,5 s trước khi đủ số episode mục tiêu: bốn barrier chờ chặn (`sr_block`, `std_barrier`, `condvar`, `pthread_barrier`) ở các pha 4, 12 và 24 thread, cùng một số pha của `sr_bo_sleep`, `sr_bo_spin` và `omp_barrier`. Riêng pha 24 thread của `sr` và `sr_bo_spin` chỉ được 70 và 59 episode. libstdc++ cài `std::barrier` ngay trong header, nên TSan thấy nó và phép kiểm handoff của `std_barrier` được chạy đủ. Bản TSan bỏ qua `omp_barrier`, cùng lý do với `omp_lock`.

4.4 Concurrent counter

`increment()` trả về giá trị trước khi tăng. Bài kiểm tra có năm phép kiểm, mang nhãn như trong Bảng 3. Sau M lần tăng, phép kiểm *final* đòi `get()` bằng M. Nhóm lưu mọi giá trị trả về. Phép kiểm *dup* tìm giá trị bị trả về hai lần, và *miss* tìm giá trị trong $\{0, \dots, M-1\}$ không được trả về. Phép kiểm *order* đòi dãy giá trị của mỗi thread tăng ngặt.

Phép kiểm *real-time* chạy ở một lượt riêng. Mỗi lần gọi được kẹp giữa hai lần `fetch_add seq_cst` lên một đồng hồ logic dùng chung, một trước lời gọi (inv) và một sau khi trả về (resp). Mọi nhịp của đồng hồ có một thứ tự toàn phần. Nếu $\text{resp}(A) < \text{inv}(B)$ thì A đã trả về trước khi B bắt đầu, nên A phải nhận giá trị nhỏ hơn.

Các phép kiểm này kiểm linearizability [4] trên các lịch chạy đã xảy ra. Tập giá trị trả về đúng là $\{0, \dots, M-1\}$ khi và chỉ khi có một thứ tự tuần tự hợp lệ. Phép kiểm *order* và *real-time* xác nhận thứ tự đó tôn trọng thời gian thực.

Mỗi counter chạy năm lượt: 1 thread, số luồng phần cứng, gấp đôi số đó, và hai lượt kiểm thời gian thực với số luồng phần cứng và gấp đôi. Ở hai lượt đầu, mỗi thread tăng 512 000 lần với

Biến thể	Lỗi cài vào	Stress (+ fuzzing)	TSan	Bị bắt bởi
`TAS_CHECK_THEN_SET`	`load` rồi `store` thay `exchange`	3/3 vi phạm loại trừ	race	cả hai
`TAS_RELEASE_RELAXED`	nhả lock bằng `store` relaxed	0/3	race	chỉ TSan
`MCS_NO_WAIT_SUCCESSOR`	nhả lock không chờ thread kế tiếp	3/3 treo	không	stress
`MCS_ALWAYS_CLEAR_TAIL`	luôn đặt `tail` về null	3/3 treo	race	cả hai
`single_flag`	một cờ, không đảo chiều	5/5 (5/5) sai episode	không	stress
`flip_before_reset`	đảo `sense` trước khi đặt lại bộ đếm	2/5 (5/5) treo	không	stress
`nonatomic_count`	`load`+`store` thay `fetch_add`	5/5 (5/5) treo	không	stress
`fetch_add_release_only`	`fetch_add` release thay acq_rel	1/5 (0/5) sai episode	race	chủ yếu TSan
`thread_local_sense`	local sense chung cho mọi barrier	5/5 (5/5) sai giá trị handoff	race	cả hai
`plain_long`	`long` thường	3/3 final, dup, miss, order	race	cả hai
`load_store`	`load` rồi `store` atomic	3/3 final, dup, miss, order	không	kiểm giá trị
`cas_no_retry`	CAS một lần, không thử lại	3/3 final, dup, miss	không	kiểm giá trị
`lock_read_outside`	đọc giá trị trước `acquire()`	3/3 final, dup, miss, order	race	cả hai
`block_reserve`	giữ trước 8 giá trị mỗi thread	3/3 chỉ real-time	không	kiểm giá trị

Bảng 3: Mutation testing trên `apple-m5`: lock (trên), barrier (giữa), counter (dưới). Cột stress cho số lần chạy bắt được trên số lần chạy (trong ngoặc: bản schedule fuzzing) cùng dấu hiệu hoặc phép kiểm báo lỗi; cột TSan cho biết TSan có báo data race không.

counter atomic, và 128 000 lần với counter dùng lock và `cas_sleep`. Ba lượt sau dùng 1/2, 1/4 và 1/8 số đó. Mỗi lượt có ngân sách 1,5 s. Hết ngân sách thì các thread dừng sớm, và phép kiểm chạy trên số lần tăng đã làm.

Ở `-O2`, compiler gộp một vòng tăng không atomic thành một lần đọc và một lần ghi. Trong một thí nghiệm phụ lúc phát triển, bài kiểm tra chỉ so giá trị cuối không phát hiện lỗi của một counter không đồng bộ ở 198/200 lần chạy với 2 thread cùng xuất phát. Với 10 thread không cùng xuất phát, nó không phát hiện lỗi ở cả 200/200 lần chạy. Bài kiểm tra của nhóm lưu mọi giá trị trả về. Nó cũng cho địa chỉ counter đi qua `pc::opaque()` ở mỗi vòng, nên mỗi lần tăng phải đọc và ghi thật.

Trên cả ba máy, `lock_mcs` hết ngân sách thời gian và chỉ được kiểm qua 1 765 312 lần tăng trên `apple-m5`, 2 049 423 lần trên `C4M7Y670NT` và 2 063 205 lần trên `LAP1184`. Trên `C4M7Y670NT`, `cas_seqcst` cũng hết ngân sách (15 602 488 lần tăng). Bản TSan trên `LAP1184` bỏ qua `lock_omp_lock`, cùng lý do với `omp_lock`.

4.5 Mutation testing

Trả lời RQ1. Không bài kiểm tra nào tìm thấy lỗi trên ba máy, và bộ kiểm tra bắt cả 14 biến thể hỏng. Tính đúng dựa trên lập luận ở mục Cài đặt. Kiểm thử củng cố lập luận đó. Trên `apple-m5`, `C4M7Y670NT` và `LAP1184`, mọi cài đặt đạt mọi bài kiểm tra (12 lock, 7 barrier và 19 counter trên hai máy Apple, 13, 9 và 20 trên `LAP1184`), và TSan không báo data race nào. Hai lỗi memory order khó bắt nhất. Stress test không bắt `TAS_RELEASE_RELAXED` lần nào (0/3 trên cả ba máy). Bản thường bắt `fetch_add_release_only` 1/5 lần trên `apple-m5`, 4/5 trên `C4M7Y670NT` và 0/5 trên `LAP1184`. TSan bắt cả hai lỗi trên mọi máy.

Bằng chứng. Bảng 3 tóm tắt kết quả trên `apple-m5`. Biến thể lock là macro sửa một chỗ trong `lock.h`. Biến thể barrier và counter là bản sao trong `tests/`, mỗi bản cài một lỗi. Riêng `single_flag` và `block_reserve` là thuật toán khác hẳn. Khi chạy biến thể hỏng, watchdog báo treo sau 5 s với lock và 2 s với barrier, và mỗi pha barrier có ngân sách 0,5 s.

Trên `C4M7Y670NT`, mọi biến thể bị bắt bởi cùng kỹ thuật như trên `apple-m5`. Chỉ số lần stress test bắt được khác. Ở bản thường, `fetch_add_release_only` bị bắt 4/5 lần (so với 1/5) và `flip_before_reset` 1/5 lần (so với 2/5). Bản fuzzing cho cùng kết quả như trên `apple-m5` (0/5 và 5/5). Kết quả TSan của máy này chỉ dựa trên dòng tóm tắt trong `mutation.log`, vì log chi tiết của các lần chạy TSan không được lưu.

Trên `LAP1184`, mọi biến thể bị bắt bởi cùng kỹ thuật như trên `apple-m5`, với ba khác biệt.

Thứ nhất, `fetch_add_release_only` không bị stress test bắt lần nào (0/5, fuzzing 0/5), chỉ TSan bắt. Thứ hai, bản thường bắt `flip_before_reset` 5/5 lần, so với 2/5 trên `apple-m5`. Thứ ba, với `thread_local_sense`, bản TSan dừng ngay khi phép kiểm handoff báo sai giá trị, và TSan không in cảnh báo nào trước đó. Trên máy này biến thể chỉ bị phép kiểm giá trị bắt (5/5 ở bản thường và bản fuzzing).

Vì sao. Hai biến thể chỉ TSan bắt ổn định đều là lỗi memory order. `TAS_RELEASE_RELAXED` đổi lệnh nhả lock `stlrb` (store release) thành `strb`. TSan bắt nó vì store relaxed không tạo cạnh happens-before giữa hai chủ lock liên tiếp. Ba lần chạy stress không có cập nhật nào bị mất. Nhóm chưa đo vì sao lỗi không lộ trên ARM. Một khả năng là các phép ghi trong critical section thường đã tới cache trước khi phép ghi nhả lock được thấy, nên cửa sổ lỗi rất hẹp. `fetch_add_release_only` đổi `ldaddal` thành `ldaddl`, nên thread tới cuối có thể đọc ô của thread khác trước khi `fetch_add` của nó có hiệu lực. TSan bắt nó qua các ô thường của phép kiểm handoff.

Trên x86-64, mọi `fetch_add` đều là lệnh `lock xadd`. Mọi store, kể cả `relaxed` và `release`, đều là `mov`. Vì vậy trên `LAP1184` hai biến thể memory order chạy đúng lệnh máy của bản đúng, và không lịch chạy nào làm lộ lỗi.

Ngược lại, TSan im lặng với bảy biến thể chỉ dùng atomic hoặc gây treo. Không kỹ thuật nào một mình bắt được mọi biến thể. Stress test hoặc phép kiểm giá trị bắt mọi lỗi thuật toán. Bốn lỗi treo (hai biến thể MCS, `flip_before_reset`, `nonatomic_count`) hiện ra qua watchdog.

Fuzzing làm `flip_before_reset` treo sau 3–7 episode (bản thường cần 39 896 và 289 052 episode ở hai lần bắt được trên `apple-m5`). Bản fuzzing không bắt được lỗi thiếu acquire (0/5). Bản này chỉ chạy một phần mười số episode, và trên hai máy Apple bản thường chỉ bắt được lỗi sau hơn 13 000 episode của một pha. Nhóm chưa đo vì sao bản thường trên `LAP1184` bắt `flip_before_reset` thường hơn. Một khả năng là máy ảo tước core của thread nhiều hơn.

Độ vững. Kết luận trên dựa trên một lượt chạy mỗi máy và chỉ trên các lịch chạy đã xảy ra. Mục 4.6 nêu các giới hạn này.

4.6 Giới hạn của phương pháp

Stress test và fuzzing chỉ xét các lịch chạy đã xảy ra. Chúng chỉ ra được là có lỗi, không chứng minh được là không có lỗi. Tính đúng vì vậy dựa trên lập luận ở mục Cài đặt.

Trên x86-64 (mô hình TSO, total store order), phần cứng chỉ đảo một phép store với phép load đứng sau. Lỗi cần giữ thứ tự này (`seq_cst`) có thể lộ qua stress test, nhưng bộ biến thể không có lỗi loại này. Với lỗi memory order, nhóm dựa chủ yếu vào TSan.

TSan chỉ tìm data race, chỉ thấy các đường đi đã chạy và chỉ thấy mã được instrument.

Bộ kiểm tra không kiểm tiến triển (lock-free, wait-free) ngoài trường hợp treo. Nó cũng không kiểm `get()` trong lúc các thread đang tăng. Các biến thể hỏng do chính nhóm chọn, nên chúng chỉ chứng tỏ bộ kiểm tra nhạy với những lỗi đó.

Mỗi máy chạy `make test`, `make tsan` và `make mutation` một lượt. Tỉ lệ bắt lỗi của bộ mutation (ví dụ 2/5 hay 1/5) vì vậy chỉ cho bậc độ lớn.

5 Phương pháp đo

Chương trình đo lock chạy vòng lặp của đề. Mỗi thread lặp I lần. Mỗi vòng lấy lock, gọi `is_prime(N)` trong critical section, nhả lock, rồi gọi `is_prime(N)` thêm OW (`OUTSIDE_WORK`) lần ở ngoài. p là số thread. Chương trình đo counter thay critical section bằng một lần tăng counter, với OW lời gọi `is_prime(101)` giữa hai lần tăng. Chương trình đo barrier gọi barrier I lần liên tiếp, không có việc ở giữa, nên mỗi lần chạy có đúng I episode.

Đồng hồ là `std::chrono::steady_clock`. Vùng đo bắt đầu khi mọi thread đã được tạo và cổng xuất phát chung mở ra, và kết thúc khi lần `join` cuối trả về. Đề yêu cầu thời gian của cả

chương trình. Báo cáo khác đề ở điểm này. Việc tạo thread nằm ngoài vùng đo, vì nó không thuộc chi phí đồng bộ mà các cài đặt cần so. Đồng hồ trên hai máy Apple có độ phân giải 41,7 ns (bộ đếm 24 MHz, Bảng 13, phụ lục). Trên `LAP1184`, một lần đọc đồng hồ tốn trung vị 20 ns, và CPU báo TSC ổn định, không đổi theo tần số. Cả hai mức này rất nhỏ so với một lần chạy, có độ dài trung vị 0,15–0,29 s tuỳ phần đo.

Mỗi lần chạy in đúng một dòng CSV gồm tham số, thời gian, load average 1 phút và kết quả của một phép kiểm. Phép kiểm buộc mọi phần việc phải thật sự chạy. Trong khung của đề, compiler được phép xoá `is_prime(N)` vì kết quả bị bỏ đi. Nhóm giữ `is_prime` không inline và cho `N` đi qua `pc::opaque()` ở mỗi lời gọi. Mỗi kết quả được cộng vào tổng riêng của thread, và tổng này phải bằng số lời gọi. Với counter, phép kiểm còn đòi giá trị cuối bằng $M = pI$ và tổng các giá trị trả về bằng $0 + 1 + \cdots + (M - 1)$. Với barrier, mọi thread phải đi hết I episode. Bảng 11 (phụ lục) xác nhận phần việc không bị xoá. Ở $N = 1009$, thời gian một lời gọi t nằm trong 504–527 ns ở mọi OW từ 0 tới 15, và t tỉ lệ với N. Trên `apple-m5`, cả 12 152 lần chạy hoàn thành đều qua phép kiểm kết quả. Các dòng không qua chỉ là lần vượt thời hạn. Trên `C4M7Y670NT` và `LAP1184`, cả 13 244 và 14 322 lần chạy hoàn thành cũng đều qua.

Phần lock quét 12 lock qua ba lượt. Một lượt dùng $N = 1009$ với OW bằng 0, 1, 3, 7, 15. Hai lượt còn lại dùng $N = 11$ và $N = 10\,007$ với OW bằng 0 và 3. N phải là số nguyên tố để `is_prime` chạy đủ $N - 2$ vòng. Ba giá trị cho critical section khoảng 5 ns, 0,5 µs và 5 µs. OW quyết định mức tranh chấp. Barrier quét 7 cài đặt. Counter quét 19 cài đặt với OW bằng 0, 1, 8. Một lượt phụ chạy lại TAS và TTAS với lệnh chờ `yield` thay `isb` ở 1, 4, 10, 20 thread, $N = 1009$, OW bằng 0 và 3 (cột Lock (`yield`) của Bảng 10). RQ3 dùng lượt này.

Số thread trên `apple-m5` là 1, 2, 3, 4, 6, 8, 10, 12, 16, 20. Trong đó 4 là số P-core, 10 là mọi core, 12–20 là quá tải. Trên `C4M7Y670NT` và `LAP1184`, lưới thêm 24 thread. Ở hai máy này, 12 là mọi core trên `C4M7Y670NT` và mọi luồng phần cứng trên `LAP1184`, còn 16–24 là quá tải. `C4M7Y670NT` có 6 P-core. `LAP1184` có thêm các baseline Linux và OpenMP, nên quét 13 lock, 9 barrier và 20 counter.

Với lock, số vòng mỗi thread là $I = W_N / \max(p, 1 + \mathrm{OW})$. W_N là tổng số vòng lặp, cộng qua mọi thread (khi $p \geq 1 + \mathrm{OW}$), để một lần chạy của `ttas` với $\mathrm{OW} = 0$ dài khoảng 0,19 s. Nhóm hiệu chỉnh W_N một lần cho mỗi N, ở 1 thread với $N = 1009$ và 10 007, ở 4 thread với $N = 11$ (ở đó chi phí chính là việc trao lock). Riêng ở $N = 11$, lần chạy 1 thread dùng I gấp 8 lần, vì W_{11} hiệu chỉnh ở 4 thread. Trên `apple-m5`, W_N bằng 5 426 304, 343 663 và 36 381 cho $N = 11$, 1009 và 10 007. Với barrier và counter, chi phí một thao tác chênh nhau nhiều bậc, nên mỗi cấu hình có I riêng suy từ một lần chạy thử, nhắm 0,25 s và 0,15 s. Trên `C4M7Y670NT`, runner thu mọi thời lượng mục tiêu lại 0,956 lần. Trên `LAP1184`, hệ số này là 0,885 (0,17, 0,22 và 0,13 s), để thời lượng ước tính còn khoảng 70 phút. I cố định cho cả đợt đo. Hình và bảng quy thời gian về cùng số vòng lặp.

Thời hạn mỗi lần chạy là 30 s cho lock và 10 s cho barrier và counter. Riêng barrier spin (`sr`, `sr_bo_spin`) khi quá tải có thời hạn 30 s, vì một episode có thể tốn cả một lượt lập lịch. Lock MCS quá tải được đo với số vòng nhỏ hơn 100 lần ngay từ đầu (27 cấu hình trên `apple-m5`). Lần vượt thời hạn được ghi lại. Cấu hình đó được đo tiếp với số vòng nhỏ hơn 100 lần (không dưới 10), và trung vị chỉ dùng các lần chạy có cùng số vòng, là số vòng của đa số lần chạy. Lần vượt thời hạn chỉ được tính là lần chậm nhất khi nó có cùng số vòng đó. Không cấu hình nào trên ba máy có trung vị là một lần vượt thời hạn.

Trên `apple-m5`, 9 lần vượt thời hạn đều là MCS quá tải (6 ở lock, 3 ở counter), và 1 trong 9 lần có cùng số vòng với các lần tính trung vị. Trên `C4M7Y670NT` có 10 lần (6 ở lock, 4 ở counter), 3 lần trong đó cùng số vòng. Trên `LAP1184` có 12 lần (6 ở lock, 6 ở counter), 1 lần cùng số vòng. Mọi lần vượt thời hạn ở lock là MCS ở $N = 11$, vốn đã chạy với số vòng giảm sẵn, nên các cấu hình này được giảm thêm 100 lần. Vì vậy vài ô MCS quá tải chỉ còn 4–6 giá trị (2 ô trên `apple-m5`, 3 trên `C4M7Y670NT`, 7 trên `LAP1184`).

Mỗi cấu hình chạy 7 lần. Báo cáo dùng trung vị, không dùng trung bình, vì phân bố lệch phải. Ở 33 % cấu hình lock, lần chậm nhất dài hơn 1,5 lần trung vị, với barrier là 49 %. Lượt lặp là vòng

ngoài. Trong mỗi lượt, mọi cấu hình chạy theo thứ tự xáo trộn với seed cố định, nên tải nền trôi chậm không thiên vị cài đặt nào. Mỗi thời điểm chỉ có một tiến trình đo, và đợt đo chỉ bắt đầu khi load average 1 phút không quá 7,0.

Phần lock được so với cận dưới lý tưởng

$$T_{\min} = \max(p,\ 1 + \mathrm{OW}) \cdot I \cdot t.$$

Từ $p = 1 + \mathrm{OW}$, lock phải bận liên tục, vì tổng thời gian critical section $p\,I\,t$ bằng hoặc vượt phần việc $(1 + \mathrm{OW})\,I\,t$ của một thread. Khi đó các critical section chạy nối tiếp, nên cần ít nhất $p\,I\,t$. Khi $p < 1 + \mathrm{OW}$, một thread cần ít nhất $(1 + \mathrm{OW})\,I\,t$ cho phần việc của riêng nó. t là trung vị lần chạy 1 thread của lock nhanh nhất ở cùng N và OW, chia cho $I(1 + \mathrm{OW})$. Báo cáo dùng hai đại lượng dẫn xuất:

- *Độ chậm* là thời gian đo chia cho $T_{\min}$. Độ chậm 1 nghĩa là lock không thêm chi phí nào.
- *Độ công bằng* là thời gian của thread xong sớm nhất chia cho thời gian của thread xong muộn nhất trong một lần chạy. Mỗi thread tự đo vòng lặp của mình. Mọi thread làm cùng số vòng, nên 1 là hoàn toàn đều. Lock và counter đều ghi đại lượng này.

Bảng 10 (phụ lục) tóm tắt đợt đo. Bảng 11 cho t.

Mỗi điểm trên hình là trung vị của 7 lần lặp, và thanh sai số đi từ min tới max. Vạch dọc liền đánh dấu số core, hoặc số luồng phần cứng khi máy có SMT, còn vạch dọc chấm đánh dấu số P-core, hoặc số core do máy ảo báo. Vòng tròn lớn quanh một điểm cho biết cấu hình đó được đo với số vòng giảm 100 lần hoặc hơn rồi quy đổi tuyến tính. Tam giác rỗng đánh dấu trung vị là một lần vượt thời hạn, và khi đó thời gian thật lớn hơn giá trị vẽ.

6 Kết quả và phân tích

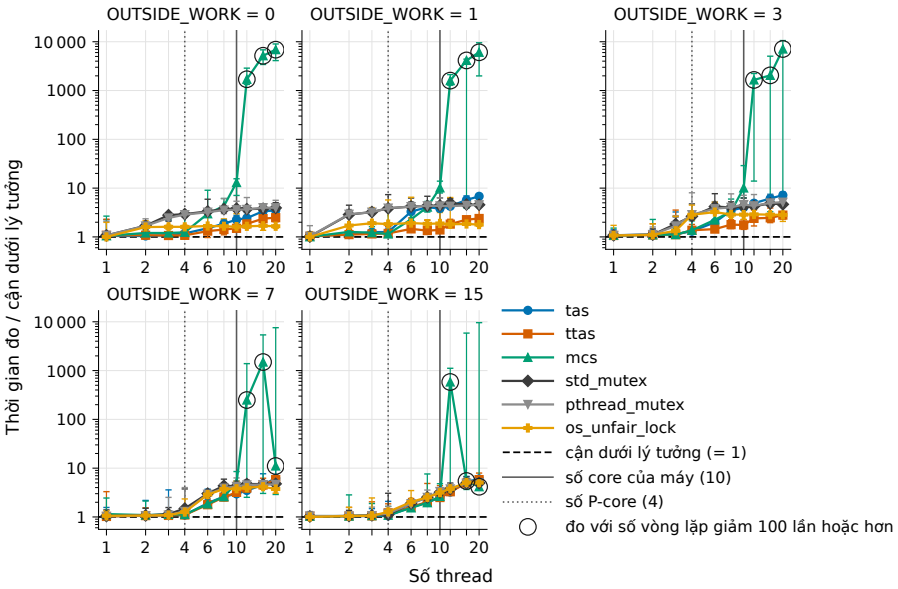
6.1 Mutex lock

Hai đại lượng dùng trong mục này là độ chậm và độ công bằng, định nghĩa ở mục Phương pháp đo.

6.1.1 RQ2: Lock mở rộng thế nào theo số thread, tranh chấp và độ dài critical section?

Trả lời RQ2. TTAS mở rộng tốt nhất trong ba lock tự viết. Ở 20 thread ($N = 1009$, $\mathrm{OW} = 0$), nó chậm 2,48 lần cận dưới. MCS gãy mạnh tại số core (13 lần ở 10 thread, 7 lần khi tải nền thấp) và sụp khi vượt số core (1 700–6 900 lần). Các ô đó đo với số vòng lặp giảm 100 lần và chỉ cho bậc độ lớn. Lock thư viện không sụp. `std_mutex` và `pthread_mutex` chậm hơn spinlock khi critical section dài cỡ 0,5 μs (3,9–4,0 lần cận dưới ở 20 thread, so với 2,48 của TTAS). Riêng `os_unfair_lock` giữ 1,6–1,7 lần. Khi critical section chỉ vài nanosecond, cả ba lock thư viện nhanh hơn spinlock không backoff từ 3 thread. Số liệu của `apple-m5`. RQ6 kiểm lại trên hai máy còn lại.

Bằng chứng. Ở $\mathrm{OW} = 0$, ba spinlock tự viết chậm không quá 1,24 lần cận dưới tới 4 thread (Hình 1). Chúng gãy từ 4 lên 6 thread (TAS 1,18 lên 1,50, TTAS 1,08 lên 1,31, MCS 1,24 lên 2,98), tức tại số P-core chứ không tại số core. MCS gãy mạnh nhất. Từ 6 thread nó chậm nhất trong ba spinlock tự viết, và từ 8 thread chậm nhất trong mọi lock (4,31 so với 3,72 của `std_mutex`). Nó chậm 13 lần ở đúng 10 thread rồi sụp hẳn khi vượt số core (Bảng 4). TAS và TTAS chỉ chậm dần, tới 3,48 và 2,48 lần ở 20 thread. Lock thư viện đã chậm 2,5–2,9 lần từ 3–4 thread (`os_unfair_lock` 1,6 lần từ 2 thread), rồi tăng chậm và gần như phẳng từ 8 thread (3,5–4,0 lần).



Hình 1: Thời gian đo chia cho cận dưới lý tưởng $\max(p\,I\,t,\ (1+\mathrm{OW})\,I\,t)$, $N = 1009$, mỗi ô một mức `OUTSIDE_WORK`. Giá trị 1 nghĩa là lock không thêm chi phí nào ngoài phần việc tuần tự bắt buộc.

Bảng 4: Trung vị thời gian chạy (s) của mọi lock trên máy `apple-m5`, $N = 1009$, `OUTSIDE_WORK` $= 0$, quy về $I = 343\,663$ vòng lặp mỗi thread (thời gian đo nhân với $I/I_{\text{đo}}$). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy. †: đo với số vòng lặp nhỏ hơn 100 lần (cấu hình sụp đổ hoặc đã vượt thời hạn), rồi quy đổi tuyến tính.

Lock \ số thread	1	2	3	4	6	8	10	12	16	20
`tas`	0,184	0,367	0,572	0,827	1,58	2,63	3,95	5,12	9,42	12,2
`tas_bo_sleep`	0,177	0,386	0,577	0,782	1,17	1,50	1,86	2,29	3,16	3,78
`tas_bo_spin`	0,189	0,373	0,554	0,774	1,35	2,24	2,75	3,62	6,27	9,11
`tas_bo_yield`	0,178	0,377	0,575	0,750	1,14	1,55	1,97	2,29	3,17	3,95
`ttas`	0,189	0,380	0,562	0,756	1,38	1,99	2,63	3,90	6,65	8,70
`ttas_bo_sleep`	0,192	0,360	0,561	0,764	1,27	1,89	2,64	3,43	6,04	9,99
`ttas_bo_spin`	0,196	0,364	0,574	0,777	1,33	1,98	2,90	3,85	6,01	9,31
`ttas_bo_yield`	0,203	0,372	0,573	0,750	1,26	1,86	2,64	3,39	5,94	7,75
`mcs`	0,185	0,422	0,633	0,872	3,13	6,04	22,8	3 590†	14 500†	24 100†
`std_mutex`	0,188	0,594	1,47	2,06	3,51	5,23	6,67	7,82	10,6	13,8
`pthread_mutex`	0,188	0,590	1,29	2,01	3,38	4,97	6,32	7,72	11,1	13,9
`os_unfair_lock`	0,175	0,558	0,848	1,13	1,76	2,32	2,84	3,48	4,74	5,81
cận dưới lý tưởng	0,175	0,351	0,526	0,702	1,05	1,40	1,75	2,11	2,81	3,51

Ô OW $= 15$ là một thí nghiệm đối chứng, vì với $p \le 15$ lock bận tối đa $p/16$ thời gian. Vậy mà ở 6 thread cả mười hai lock, kể cả lock thư viện, cùng chậm 1,53–2,03 lần cận dưới.

Từ $p = 1 + \mathrm{OW}$, lock phải bận liên tục thì mới đạt cận dưới, nên mỗi lần chuyển giao chậm đều cộng thẳng vào thời gian chạy. Lock thư viện cho thấy điểm này rõ nhất. Ở OW $= 1$ và 2 thread, `std_mutex` và `pthread_mutex` đã chậm 2,9 lần cận dưới (`os_unfair_lock` 1,7 lần), còn ba spinlock tự viết chậm 1,11–1,27 lần. Ở OW $= 3$, ba lock thư viện chậm 1,34–1,83 lần ở 3 thread và 2,64–2,86 lần ở 4 thread, còn spinlock 1,11–1,13 và 1,31–1,37 lần. Nhóm cho rằng thread chờ của lock thư viện ngủ, nên mỗi lần chuyển giao trả thêm một lần đánh thức qua kernel (ước lượng ở dưới), và lock thư viện bão hoà sớm hơn spinlock. Với OW $= 7$ và 15, chỗ gãy từ 4 lên 6 thread tới trước điểm bão hoà (TAS 1,09 lên 1,92 và 1,05 lên 1,59), và mọi lock tăng cùng nhau.

Khi vượt số core ở OW $= 15$, riêng phần việc đã vượt sức máy. Tổng $p(1 + \mathrm{OW})\,I$ lời gọi `is_prime` chia cho 10 core cần ít nhất $p(1 + \mathrm{OW})\,I\,t/10$, tức 1,6 lần cận dưới ở 16 và 20 thread,

trước khi tính chi phí lock. TAS, TTAS và lock thư viện cùng nằm trong 4,9–6,0 lần ở 16–20 thread. Nhóm cho rằng phần còn lại đến từ E-core chậm hơn và từ việc chia lượt lập lịch.

Với $N = 10\,007$ (khoảng 5,2 μs) TAS và TTAS gần như trùng nhau (Hình 10, phụ lục). Với $N = 11$ (4,6 ns), ở 20 thread TAS chậm 80,5 lần cận dưới, TTAS 28,2 lần, còn lock thư viện chỉ khoảng 5 lần.

Vì sao. Nhóm cho rằng chỗ gãy ở 4 thread đến từ máy, không từ thuật toán. Ở ô đối chứng, mọi lock cùng chậm như nhau. Độ công bằng ở 6 thread là 0,84–0,98, tức mọi thread cùng chậm đi. Điều này phù hợp với việc scheduler luân chuyển thread giữa P-core và E-core chậm hơn, hoặc với tải nền (xem mục Các yếu tố đe dọa độ tin cậy). Trên `C4M7Y670NT` (6 P-core, 12 core), điểm gãy dời sang 6 lên 8 thread và MCS sụp khi vượt 12 thread (mục So sánh giữa các máy).

MCS gãy mạnh ngay tại số core và sụp khi vượt số core, sớm hơn mức mà riêng lưu lượng cache coherence giải thích được. Nguyên nhân nằm ở cách trao lock. MCS trao lock cho đúng một thread kế tiếp theo thứ tự FIFO (vào trước được phục vụ trước). Nếu thread đó không có core, lock nằm yên cho tới khi scheduler cho nó chạy lại, và cả hàng chờ mất một lượt lập lịch. TAS và TTAS không bị như vậy, vì thread nào đang chạy cũng lấy được lock rảnh. Độ công bằng của MCS bằng 1 tới 10 thread (Hình 3), tức lock luôn đi đúng thứ tự hàng đợi.

Chương trình `handoff_gap` kiểm cơ chế này. Nó chạy đúng vòng lặp của phần lock ($N = 1009$, $\mathrm{OW} = 0$) cho MCS, TAS, TTAS và `std_mutex` ở 4, 8, 10 và 12 thread. Mỗi lần chạy có khoảng 40 000 critical section (3 000 cho MCS ở 12 thread), và mỗi cấu hình chạy 5 lần theo thứ tự xáo trộn. Thread nhả lock ghi lại thời điểm nhả, và thread lấy lock kế tiếp đo *khoảng trống trao lock*, tức thời gian từ lúc lock được nhả tới lúc nó có chủ mới. Ở 4 thread, trung vị của khoảng trống dài nhất trong một lần chạy là 6–22 μs với cả bốn lock. Vì vậy một khoảng trống dài hơn 100 μs được coi là lock đã chờ một thread không có core. Ở 4 thread, chỉ một trong 20 lần chạy có khoảng trống như vậy (MCS, hai lần trao lock).

Bảng 5 cho thấy MCS ở 10 thread chỉ có 0,25 % lần trao lock dài như vậy, mỗi lần trung bình 0,79 ms, nhưng chúng chiếm 47 % thời gian chạy. Ở 12 thread, 27 % lần trao lock là lần dài và chúng chiếm 99 % thời gian. TAS, TTAS và `std_mutex` gần như không có lần trao lock dài nào. Số lần chuyển ngữ cảnh không tự nguyện của tiến trình MCS tăng từ 60 ở 4 thread lên 1 970 ở 10 thread và 23 044 ở 12 thread. Phép đo này chạy khi load average khoảng 2,8, nên ngay ở 10 thread đã có thread bị tước core. Đối chứng là TAS ở 10 thread. Tiến trình TAS cũng bị chuyển ngữ cảnh 1 022 lần, cùng bậc với MCS, nhưng trung vị số lần trao lock dài bằng 0, vì thread nào đang chạy cũng lấy được lock rảnh. Phép đo này không kiểm điểm gãy từ 4 lên 6 thread, nên cơ chế của điểm gãy đó vẫn là giả thuyết.

	4 thread	8 thread	10 thread	12 thread
`mcs`	0 / 0	0,057 / 8,3	0,25 / 47	27 / 99
`tas`	0 / 0	0 / 0	0 / 0	0,003 / 0,3
`ttas`	0 / 0	0 / 0	0 / 0	0 / 0
`std_mutex`	0 / 0	0 / 0	0 / 0	0 / 0

Bảng 5: Lần trao lock dài hơn 100 μs trên `apple-m5`, $N = 1009$, $\mathrm{OW} = 0$: tỉ lệ trên mọi lần trao lock (%) / phần thời gian chạy nằm trong các lần đó (%). Trung vị của 5 lần chạy.

Cùng giả thuyết phù hợp với việc MCS gãy mạnh hơn TAS và TTAS từ 4 lên 6 thread. MCS buộc lock đi qua mọi thread, kể cả thread đang chạy trên E-core hay đang chờ core. TAS và TTAS cho phép thread vừa nhả lấy lại lock. Ở 4 thread độ công bằng của chúng là 0,49 và 0,26, của MCS là 1,00.

Lock thư viện cho thread chờ ngủ trong kernel, nên thread chờ không giành core của chủ lock và lock không sụp. Thread đang ngủ cũng không đọc cache line của lock, nên thêm thread chờ gần như không thêm chi phí. Một critical section của `std_mutex` tốn 1 901 ns ở 8 thread và 2 007 ns ở 20 thread.

Nhóm giả thuyết rằng mỗi lần đánh thức một thread qua kernel tốn một chi phí cố định cỡ 1,5 μs. Con số này là ước lượng suy ra từ chính số đo lock. Báo cáo không có phép đo đánh thức riêng. Ở $N = 1009$, mỗi critical section của `std_mutex` ở 20 thread tốn thêm 1 459 ns so với khi không tranh chấp (548 ns ở 1 thread, Bảng 12, phụ lục). Ở $N = 10\,007$, mức tăng là 1 469 ns, gần như không đổi dù phần việc dài gấp 10 lần. Mức tăng đó bằng khoảng 0,28 lần một lời gọi `is_prime(10 007)`, khớp với mức 1,25–1,41 lần cận dưới của lock thư viện từ 4 thread ở $N = 10\,007$. Nhóm cho rằng `pthread_mutex` (và `std_mutex` dựng trên nó) trên macOS trao lock cho thread được đánh thức, nên gần như mỗi lần chuyển giao trả trọn một lần đánh thức. Tới 10 thread, độ công bằng 0,88–0,97 của hai lock này phù hợp với điều đó. Khi vượt số core, nó giảm dần xuống 0,78–0,79 ở 20 thread. Nhóm cho rằng khi đó scheduler chia core không đều giữa các thread, độc lập với lock. Ở 2 thread mức tăng mới là 316 ns. Có thể thread chờ duy nhất nhiều lần chưa kịp ngủ khi lock được nhả.

Tài liệu của `os_unfair_lock` (`os/lock.h`) ghi rằng thread vừa nhả có thể lấy lại lock trước khi thread được đánh thức kịp thử. Phần lớn critical section vì thế không phải trả lần đánh thức nào. Một critical section chỉ tốn thêm 302–351 ns từ 2 tới 20 thread, và độ công bằng giảm từ 0,97 ở 3 thread xuống 0,20 ở 20 thread.

Ở $N = 11$, `std_mutex` tốn thêm 19,7 ns mỗi critical section. Nếu mỗi lần đánh thức tốn 1 459 ns như ở $N = 1009$, một lần đánh thức được chia cho khoảng 74 critical section. Nhóm cho rằng trong lúc thread được đánh thức chưa kịp chạy, thread vừa nhả lock lấy lại lock và chạy tiếp các critical section của mình với cache line còn nóng. Chuỗi này kéo dài khoảng 1,9 μs, cùng bậc với lần ngủ ngắn nhất của `sleep_for` khi yêu cầu 100 ns (2 540 ns, Bảng 13, phụ lục). Chi phí thêm mỗi critical section nhỏ hơn một critical section của TAS khi tranh chấp (39,5 ns ở 4 thread, gồm 4,6 ns của `is_prime(11)`), nên phần lớn critical section không thể có chuyển giao. Độ công bằng 0,92 của `std_mutex` ở 20 thread không chống lại giả thuyết. Thước đo này không thấy các đợt độc chiếm luân phiên (mục Các yếu tố đe dọa độ tin cậy).

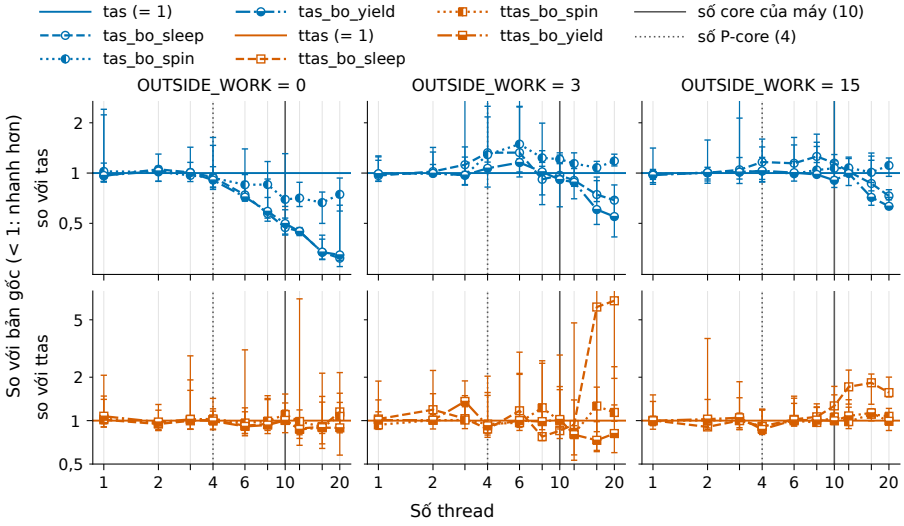
TAS thua TTAS nhiều nhất khi lock đổi chủ ở mỗi lượt. Phép nhả lock của TAS phải tranh cache line với các `exchange` liên tục của thread chờ, còn thread TTAS chờ bằng phép đọc. Ở OW = 1 và 8 thread, TAS đạt 0,467 triệu lần lấy lock mỗi giây, TTAS 1,41 triệu. Khi không có việc ngoài, lock chủ yếu ở lại với thread vừa nhả. Độ công bằng của TAS 10 thread là 0,32 ở OW = 0 và 0,91 ở OW = 3. Vì vậy TAS ở OW = 1 và 3 chậm hơn ở OW = 0 (6,79 và 7,17 so với 3,48 lần cận dưới ở 20 thread).

Độ dài critical section quyết định tỉ lệ giữa chi phí chuyển giao và phần việc (Hình 10, phụ lục). Ở $N = 11$ và 4 thread, một critical section của TAS tốn 39,5 ns, gấp khoảng 8,6 lần lời gọi `is_prime(11)`. MCS chậm nhất ở điểm này (86 ns mỗi critical section, 18,7 lần cận dưới). Nhóm cho rằng thứ tự FIFO buộc mọi critical section đổi chủ, còn TAS và TTAS cho phép thread vừa nhả lấy lại lock thành từng đợt. Ở $N = 10\,007$, chi phí chuyển giao của TAS chưa tới 1 % của 5,2 μs. Tới 4 thread mọi lock nằm trong 1,0–1,3 lần cận dưới. Từ 6 thread TAS và TTAS tách dần (1,56 và 1,57 lần ở 10 thread). Khi vượt số core, khác biệt quay lại rõ. TAS và TTAS chậm 2,5 lần ở 20 thread, lock thư viện 1,3 lần, còn MCS sụp. Nhóm cho rằng thread chờ quay giành core của chủ lock, còn thread chờ của lock thư viện thì ngủ.

Khi vượt số core, một critical section của MCS tốn gần như cùng thời gian ở $N = 1009$ và $N = 10\,007$ (3,5 và 3,3 ms ở 20 thread), dù phần việc dài gấp 10 lần. Chi phí đó nằm ngoài phần việc. Độ chậm của MCS ở $N = 10\,007$ (642 lần) thấp hơn ở $N = 1009$ (6 880 lần) chỉ vì cận dưới lớn hơn.

Độ vững. Cả bảy lần chạy MCS 10 thread đều chậm 7,1–15,5 lần cận dưới. Khi đo lại với load average khoảng 3 thay vì khoảng 25, chỉ MCS ở 10 thread nhanh lên rõ, gần gấp đôi. Thời gian của các lock khác bằng 0,83–1,05 lần thời gian ở đợt đo chính (mục Các yếu tố đe dọa độ tin cậy). Điều này phù hợp với cơ chế trên.

Các điểm MCS vượt số core ở $N = 11$ không đo được sự sụp đổ, vì sau lần vượt thời hạn số vòng lặp giảm tới mức hàng đợi không kịp hình thành. Ở OW ≥ 1, một số ô MCS vượt số core cho kết quả hai cực, nên chỉ khoảng min–max đáng tin. Ở OW ≥ 7, trung vị ở 20 thread rơi vào



Hình 2: Thời gian của từng biến thể có backoff chia cho lock gốc không backoff (đường ngang ở 1), $N = 1009$; hàng trên là họ TAS, hàng dưới là họ TTAS. Kiểu đường cho biết chính sách chờ: ngủ bằng `sleep_for` (`_bo_sleep`), quay (`_bo_spin`), quay rồi nhường core (`_bo_yield`).

cực nhanh. Ở 16 thread, OW = 15, bốn lần chạy xong dưới 10 ms, ba lần mất 2,0–10,6 s. Nhóm cho rằng khi phần lớn thời gian nằm ngoài lock, hàng chờ thường ngắn. Lần chạy nào không có lượt trao lock cho thread mất core thì xong nhanh, lần chạy nào có thì kéo dài nhiều giây.

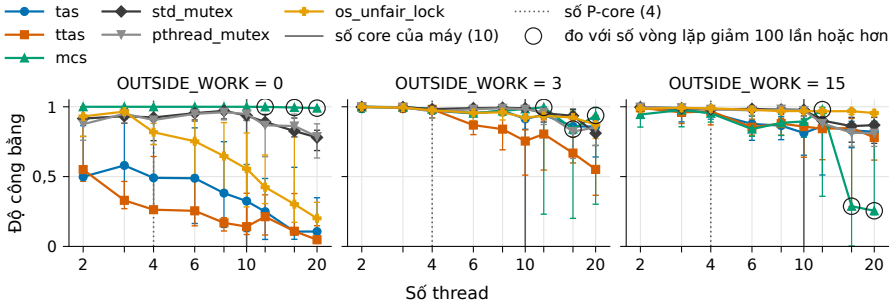
6.1.2 RQ3: Backoff giúp hay hại, và lock nhanh nhất có công bằng không?

Trả lời RQ3. Backoff giúp TAS và gần như không đổi gì với TTAS. Ở 20 thread ($N = 1009$, OW = 0), backoff bằng `sleep_for` đưa TAS từ 12,2 s xuống 3,78 s (thời gian quy về cùng số vòng lặp). Lợi ích đến từ việc lock gần như không đổi chủ, nên lock nhanh nhất lại kém công bằng nhất. Khi có việc ngoài critical section và tranh chấp vừa phải, backoff làm chậm. Hại lớn nhất là ở `ttas_bo_sleep` khi vượt số core với OW = 3, chậm hơn TTAS 6,1–6,8 lần. Số liệu của `apple-m5`. RQ6 kiểm lại trên hai máy còn lại.

Bằng chứng. Ở OW = 0 (Hình 2, cột trái), `tas_bo_sleep` và `tas_bo_yield` chỉ mất 0,31–0,32 lần thời gian của TAS ở 20 thread, còn `tas_bo_spin` mất 0,75 lần. Các biến thể của TTAS nằm trong 0,87–1,15 lần TTAS. Ở OW = 3 và 4–6 thread, backoff làm TAS chậm hơn (`tas_bo_sleep` 1,32–1,33 lần, `tas_bo_spin` 1,29–1,49 lần). Ở 20 thread, `tas_bo_sleep` và `tas_bo_yield` lại nhanh hơn (0,69 và 0,55 lần TAS), còn `tas_bo_spin` vẫn chậm hơn (1,18 lần). Trên `LAP1184`, cũng ở OW = 3, sleep backoff làm TAS chậm hơn cả ở số thread lớn (RQ6). Ở OW = 15, mọi biến thể nằm trong 0,90–1,26 lần TAS tới 12 thread. Ở 20 thread `tas_bo_sleep` và `tas_bo_yield` nhanh hơn (0,73 và 0,63 lần).

Ở OW = 0, MCS giữ độ công bằng 1 tới 10 thread (Hình 3). Độ công bằng của `tas_bo_sleep` bằng đúng $1/p$, tới 0,05 ở 20 thread, tức là các thread chạy xong lần lượt từng thread một. TAS và TTAS cũng thiên vị thread vừa nhả lock. TTAS thiên vị hơn TAS (0,26 so với 0,49 ở 4 thread, 0,05 so với 0,11 ở 20 thread). Nhóm cho rằng phép nhả của TTAS không phải tranh cache line với `exchange` nào, nên thread vừa nhả lấy lại lock trong lúc các thread chờ còn đang đọc lại line. `os_unfair_lock`, lock thư viện nhanh nhất, kém công bằng nhất trong ba lock thư viện (0,20 so với 0,78–0,79 ở 20 thread), vì lý do đã nêu ở RQ2.

Ở OW = 3, mọi lock gần như công bằng tới 4 thread. Từ 6 thread, p vượt $1 + \text{OW}$ nên luôn



Hình 3: Độ công bằng, $N = 1009$: thời gian của thread xong sớm nhất chia cho thread xong muộn nhất trong cùng một lần chạy (1 là hoàn toàn đều). Cấu hình có trung vị vượt thời hạn không được vẽ.

có thread chờ, và cùng sự thiên vị đó làm độ công bằng của TTAS giảm từ 0,98 ở 4 thread xuống 0,87 ở 6 thread và 0,55 ở 20 thread.

Khi OW > 0, thread vừa nhả đi gọi `is_prime` thay vì lấy lại lock ngay. Vì vậy `os_unfair_lock` gần công bằng ở OW $= 3$ và 15 (0,87–1,00), dù ở OW $= 0$ nó chỉ còn 0,20 ở 20 thread.

Vì sao. Các biến thể `tas_bo_*` lùi lại sau mọi lần `exchange` thất bại, kể cả khi lock chỉ đang bị giữ, nên thread chờ gần như luôn ngủ hoặc nhường core. Các biến thể `ttas_bo_*` chỉ lùi lại sau một lần `exchange` thua và vẫn đọc liên tục như TTAS (mục Cài đặt). Một lần `sleep_for` yêu cầu 100 ns thật ra ngủ trung vị 2 670 ns. Ở OW $= 0$, chủ lock nhả rồi lấy lại lock ngay ở lượt kế tiếp của vòng lặp. Thread vừa thức dậy gần như luôn thấy lock bận và ngủ tiếp với thời gian yêu cầu gấp đôi, tới khoảng 100 μs (`MAX_DELAY` trong `backoff.hpp`). Vì vậy một thread giữ lock tới hết vòng lặp của nó, rồi mới tới thread khác. Độ công bằng đúng bằng $1/p$ phù hợp với điều này.

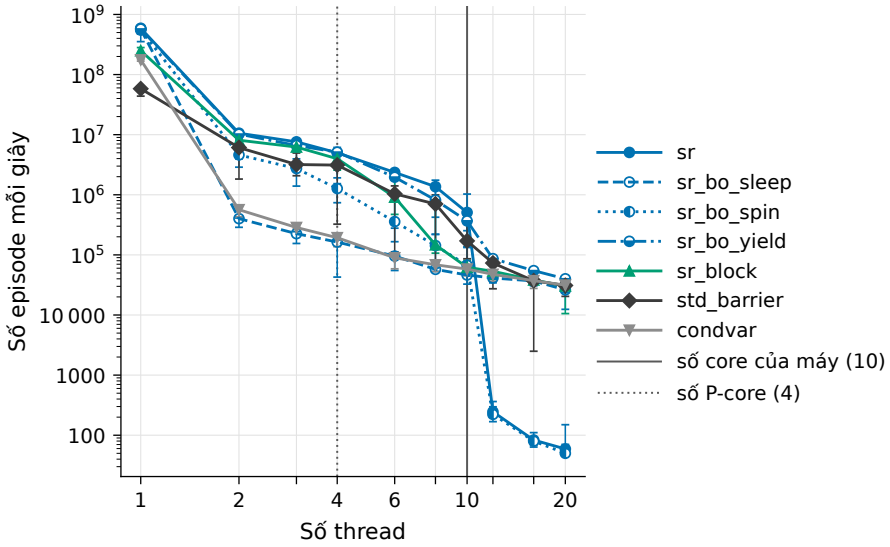
`tas_bo_spin` chỉ giãn các lần thử. Thread chờ vẫn giữ core và vẫn gọi `exchange` sau mỗi cửa sổ quay (tối đa 1 024 lần `cpu_relax()`, `backoff.hpp`). Nhóm cho rằng vì vậy lợi ích của nó nhỏ hơn, và khi vượt số core nó còn chiếm core mà chủ lock cần.

Khi OW > 0, lock rảnh trong lúc thread chờ còn ngủ ít nhất 2,5 μs. Với `tas_bo_spin`, cửa sổ quay lớn nhất kéo dài khoảng 10 μs (1 024 lần `cpu_relax()`, mỗi lần trung vị 9,6 ns), và lock cũng rảnh trong lúc đó. Ở tranh chấp vừa phải, đó là thời gian mất. Ở số thread lớn, luôn có thread sẵn sàng lấy lock. Ở OW $= 3$ và 15, khi vượt số core, thread ngủ hoặc nhường core để lại core cho thread đang làm việc ngoài critical section. Nhóm cho rằng đó là lý do `tas_bo_sleep` và `tas_bo_yield` nhanh hơn TAS ở 20 thread, còn `tas_bo_spin`, vẫn giữ core, thì không.

MCS chia lượt đều khi mọi thread được chạy, và cả khi vượt số core nếu OW ≤ 3 (độ công bằng 0,84–1,00). Khi vượt số core và OW ≥ 7, MCS kém công bằng nhất trong mười hai lock (0,21–0,30 ở 16–20 thread, so với 0,78–0,81 của lock kế tiếp). Nhóm cho rằng một thread MCS bị lấy core giữa `release` và `acquire` không nằm trong hàng đợi, nên các thread còn chạy lấy lock nhiều lượt hơn. Khi OW lớn, khoảng giữa `release` và `acquire` dài, nên điều này xảy ra thường hơn.

Khi vượt số core và OW ≥ 3, `ttas_bo_sleep` chậm hơn TTAS 6,1–6,8 lần ở OW $= 3$ với 16–20 thread, 2,0–4,3 lần ở OW $= 7$ và 1,6–1,8 lần ở OW $= 15$ (12–20 thread). Nhóm giả thuyết rằng ở biến thể này một phần thread chờ quay đọc và chiếm core, phần khác ngủ. Thread vừa thức dậy sau khi ngủ có thể giành core của chủ lock. Trong lúc chủ lock không chạy, các thread quay đọc dùng hết lượt của mình. `ttas_bo_yield`, chỉ khác ở chỗ nhường core thay vì ngủ, không chậm hơn TTAS (1,97 s so với 2,42 s ở 20 thread, OW $= 3$, thời gian quy về cùng số vòng lặp). `tas_bo_sleep`, nơi mọi thread chờ đều ngủ, cũng không bị. Hai số đo này phù hợp với giả thuyết.

Độ vững. Kết quả của `ttas_bo_sleep` khi vượt số core phân tán mạnh giữa các lần lặp. Ở 16 thread và OW $= 3$, thời gian quy về cùng số vòng lặp đi từ 1,0 tới 18,0 s, với trung vị 10,4 s. Đổi lệnh chờ `isb` thành `yield` không tạo khác biệt vượt độ phân tán.



Hình 4: Số barrier episode mỗi giây theo số thread, không có việc giữa hai barrier (trục tung logarit). `sr` chờ bằng vòng spin, `sr_bo_*` thêm backoff, `sr_block` chờ chặn.

6.2 Barrier

6.2.1 RQ4: Chi phí một barrier episode

Trả lời RQ4. Với chờ spin, một episode tốn thêm khoảng 53 ns cho mỗi thread trong cụm 4 P-core. Độ dốc tăng mạnh khi vượt 4 thread (số P-core), gần gấp 6 theo trung vị. Với `sr`, số episode mỗi giây giảm khoảng 2 000 lần khi số thread vượt số core. Khi mỗi thread có core riêng, spin thuần nhanh nhất hoặc ngang `sr_bo_yield` (200 so với 195 ns ở 4 thread). Khi vượt số core, `sr_bo_yield` nhanh nhất. Số liệu của `apple-m5`. RQ6 kiểm lại trên hai máy còn lại.

Bằng chứng. Nhóm đo số episode (một lần cả p thread cùng qua barrier) mỗi giây khi giữa hai barrier không có việc gì (Hình 4, Bảng 6).

Bảng 6: Trung vị số barrier episode mỗi giây trên máy `apple-m5` theo số thread (k = nghìn, M = triệu; t/h = vượt thời hạn).

Barrier \ số thread	1	2	3	4	6	8	10	12	16	20
`sr`	589 M	10,6 M	7,60 M	5,00 M	2,36 M	1,36 M	508 k	246	83,2	59,1
`sr_bo_sleep`	572 M	401 k	226 k	164 k	95,6 k	57,5 k	46,3 k	41,3 k	36,8 k	26,2 k
`sr_bo_spin`	550 M	4,61 M	2,73 M	1,27 M	358 k	143 k	67,2 k	222	79,8	50,0
`sr_bo_yield`	573 M	10,2 M	6,61 M	5,12 M	1,96 M	817 k	368 k	86,5 k	54,7 k	40,0 k
`sr_block`	250 M	8,09 M	6,22 M	3,98 M	913 k	147 k	62,3 k	53,3 k	38,2 k	30,4 k
`std_barrier`	58,1 M	6,10 M	3,19 M	3,12 M	1,04 M	703 k	171 k	73,2 k	37,1 k	31,0 k
`condvar`	176 M	565 k	286 k	193 k	89,5 k	68,4 k	58,1 k	47,5 k	37,3 k	31,4 k

Từ 1 lên 2 thread, một episode của `sr` tăng từ 1,7 ns lên 94,5 ns. Từ 2 tới 4 thread, mỗi thread thêm vào tốn khoảng 53 ns. Từ 4 tới 10 thread, mức này là khoảng 295 ns. Từ 10 lên 12 thread, `sr` chậm đi khoảng 2 070 lần. Mỗi thread vượt số core thêm khoảng 2 ms vào một episode. `sr_bo_spin` sụp cùng chỗ (222 episode mỗi giây ở 12 thread).

Tới 4 thread, `sr`, `sr_block` và `sr_bo_yield` gần nhau. `std_barrier` chậm hơn `sr` 1,6 lần, còn `sr_bo_spin`, `condvar` và `sr_bo_sleep` chậm hơn 3,9–31 lần. Từ 6 tới 10 thread, các barrier chờ

chặn chậm hẳn so với `sr`. Khi vượt số core, thứ tự đảo lại. Năm barrier không quay thuần (mọi barrier trừ `sr` và `sr_bo_spin`) nhanh hơn `sr` khoảng 170–680 lần, và `sr_bo_yield` nhanh nhất.

Vì sao. Ở 1 thread, bộ đếm và cờ nằm yên trong cache của một core. Từ 2 thread, cả hai cache line phải đi giữa các core ở mỗi episode. Phần tuyến tính đến từ p phép `fetch_add` lần lượt trên cache line của bộ đếm và $p-1$ lần đọc lại cờ sau khi cờ đảo. Chỗ gãy sau 4 thread trùng với số P-core. Kết quả lock cũng gãy ở cùng chỗ, ngay cả ở ô đối chứng mà lock gần như rảnh (RQ2). Vì vậy nhóm cho rằng nó đến từ máy chứ không từ thuật toán barrier. Hai nguyên nhân có thể là E-core chậm hơn và cache line phải đi giữa hai cụm L2 (xem mục Các yếu tố đe dọa độ tin cậy).

Một episode cần mọi thread được chạy. Nhóm cho rằng khi vượt số core, thread đã tới barrier thì quay và giữ core tới hết lượt lập lịch, nên thread chưa tới phải chờ lượt sau mới được chạy. Barrier giống MCS ở điểm này (RQ2). Nhóm chưa đối chiếu con số 2 ms với độ dài lượt lập lịch của macOS. `sr_bo_spin` sụp cùng chỗ vì thread chờ của nó cũng giữ core.

`sr_block` chờ bằng `std::atomic::wait`. Trong libc++ của Apple clang 21, lời gọi này đọc 64 lần, đọc tiếp tới khi đã chờ 4 μs, rồi mới ngủ trong kernel. Tới 4 thread, episode ngắn hơn 4 μs nhiều, nên `sr_block` gần bằng `sr` (124–251 so với 94–200 ns). Từ 6 thread nó chậm hẳn (1,1 μs ở 6, 6,8 μs ở 8, 16,1 μs ở 10 thread). Nhóm cho rằng ngày càng nhiều episode có thread chờ quá 4 μs, ngủ, rồi phải được đánh thức qua kernel. Điểm đổi chiều so với `sr` nằm giữa 10 và 12 thread. Ở 10 thread, `sr_block` (16,1 μs) vẫn chậm hơn `sr` (1,97 μs). Ở 12 thread, nó nhanh hơn khoảng 217 lần.

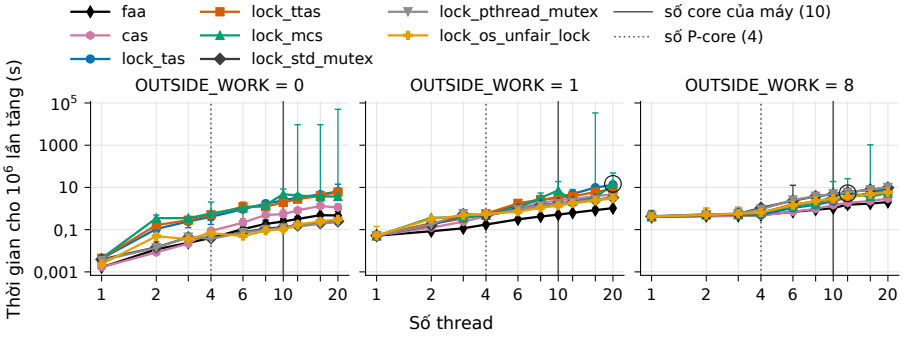
`sr_bo_yield` đọc cờ 64 lần rồi mới gọi `sched_yield` (`backoff.hpp`). Tới 4 thread, nó ngang `sr`. Từ 6 thread, có lần chờ đủ dài để gọi `sched_yield`, và lời gọi hệ thống này làm thread chậm nhận ra cờ đã đảo, nên `sr_bo_yield` chậm hơn `sr` tới 1,7 lần (8 thread). Khi vượt số core, spin sụp ngay và barrier chờ chặn thắng spin. Nhưng `sr_bo_yield` (không chặn) mới nhanh nhất. Ở 12 thread, một episode của nó tốn 11,6 μs, so với 13,7 μs của `std_barrier` và 18,8 μs của `sr_block`. Thread chờ nhường core, có thể cho thread chưa tới barrier (`sched_yield` không bảo đảm điều này). Nhờ vậy episode xong mà không phải chờ hết lượt lập lịch, và không phải trả một lần đánh thức qua kernel như `sr_block`.

Hàm `wait` của `std_barrier` trong libc++ đọc tới khi đã chờ 4 μs, gọi `yield` tới 64 μs, sau đó mới ngủ bằng `sleep_for`. Vì vậy từ 6 tới 10 thread nó chậm đi ít hơn `sr_block` (5,9 so với 16,1 μs ở 10 thread), và khi vượt số core nó gần `sr_bo_yield`. `sr_bo_yield` nhanh hơn `std_barrier` ở mọi số thread từ 2 tới 20. Ở 1 thread, một episode của `std_barrier` tốn 17 ns so với 1,7 ns của `sr`. Tới 4 thread nó chậm hơn `sr` 1,6–2,4 lần. Điều này phù hợp với việc phép tới barrier của libc++ đi qua một hàm thư viện (`__arrive_barrier_algorithm_base`) nặng hơn một lệnh `fetch_add`. Từ 3 lên 4 thread, một episode của `std_barrier` gần như không đổi (314 và 321 ns). Ở 3 thread các lần lặp dao động 202–482 ns, nên nhóm không diễn giải bước này.

`condvar` không có pha quay. Mọi thread chờ ngủ trong `pthread_cond_wait`, và thread tới cuối đánh thức tất cả bằng `notify_all`. Vì vậy ngay ở 2 thread, mỗi episode đã tốn 1,77 μs, so với 124 ns của `sr_block`. Tới 4 thread, mỗi thread thêm vào tốn khoảng 1,7 μs. Điều này phù hợp với việc mỗi thread chờ trả một lần đánh thức qua kernel, cùng bậc với chi phí 1,5 μs ước lượng ở RQ2. Ở 1 thread, `condvar` tốn 5,7 ns so với 1,7 ns của `sr`, vì mỗi lần tới barrier vẫn lấy và nhả một mutex.

Khi vượt số core, các barrier không quay thuần hội tụ về 18–38 μs mỗi episode ở 16–20 thread, tức 1,1–1,9 μs cho mỗi thread. Nhóm cho rằng mỗi episode phải lập lịch lại từng thread một lần, và chi phí đó lấn át khác biệt giữa các cách chờ.

Backoff làm barrier chậm đi khi mỗi thread có core riêng (`sr_bo_spin` chậm hơn `sr` 2,3 tới 9,5 lần). Thread chờ chỉ đọc cờ, nên backoff không có tranh chấp nào để giảm. Cửa sổ quay của `sr_bo_spin` nhân đôi sau mỗi lần đọc thấy cờ chưa đảo, tới 1 024 lần `cpu_relax()` (`backoff.hpp`). Thread chờ càng lâu thì càng nhận ra cờ đảo muộn, nên khoảng cách với `sr` tăng theo số thread tới 8 thread (2,3 lần ở 2 thread, 9,5 lần ở 8 thread). Một episode của `sr_bo_sleep` ở 2 thread tốn 2,49 μs, xấp xỉ thời gian ngủ thật ngắn nhất của `sleep_for` khi yêu cầu 100 ns (2,54 μs, Bảng 13,



Hình 5: Thời gian chạy quy về 10^6 lần tăng mỗi thread, mỗi ô một lượng việc `OUTSIDE_WORK` (lời gọi `is_prime(101)`) giữa hai lần tăng. Các biến thể có backoff và `seq_cst` chỉ có trong bảng thời gian của counter.

phụ lục). Từ đó nó tăng lên 21,6 μs ở 10 thread. Nhóm cho rằng episode phải chờ thread ngủ muộn nhất, và thời gian ngủ yêu cầu nhân đôi sau mỗi lần thức dậy thấy cờ chưa đảo.

Độ vững. Độ dốc trong câu trả lời tính từ trung vị. Giữa 7 lần lặp, một episode của `sr` ở 4 thread dao động 163–340 ns. Tính riêng cho từng lượt lặp, độ dốc từ 4 tới 10 thread vẫn lớn hơn độ dốc từ 2 tới 4 thread ở cả 7 lượt, nhưng gấp 1,2 tới 10,7 lần. Vì vậy hướng tăng của độ dốc là chắc chắn, còn hệ số gần 6 thì không. Các ô sụp đổ chỉ cho biết bậc độ lớn (mục Các yếu tố đe dọa độ tin cậy). Khoảng cách 2–3 bậc vẫn lớn hơn nhiều độ phân tán giữa 7 lần lặp. Thanh sai số dài của `std_barrier` ở 16 thread đến từ hai lần lặp chậm 5 và 15 lần so với trung vị. Nhóm chưa giải thích hai lần này.

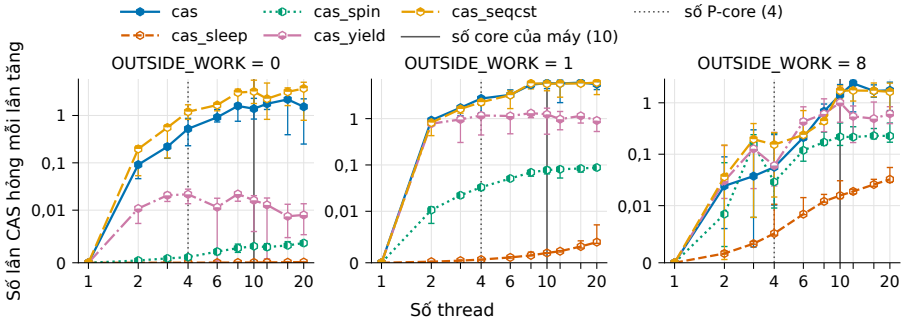
6.3 Concurrent counter

6.3.1 RQ5: Thứ tự hiệu năng của các counter

Trả lời RQ5. Khi không có việc giữa hai lần tăng, fetch-and-add nhanh hơn CAS 2,0 tới 2,7 lần từ 4 thread, vì CAS hỏng và phải thử lại. Counter dùng spinlock không backoff chậm nhất. Ở chế độ này, từ 4 thread, `lock_std_mutex` và `lock_pthread_mutex` ngang hoặc nhanh hơn fetch-and-add. CAS có backoff nhanh hơn fetch-and-add từ 2 thread. CAS với sleep backoff cho các thread tăng lần lượt (độ công bằng xấp xỉ $1/p$). `cas_spin` và `cas_yield` chỉ dồn một phần các lần tăng về một thread (độ công bằng 0,52 và 0,39 ở 4 thread). Với mutex thư viện, cơ chế tương tự mới là giả thuyết. Lợi thế này mất khi có việc giữa hai lần tăng. Số liệu của `apple-m5`. RQ6 kiểm lại trên hai máy còn lại.

Nhóm đo thời gian chạy, quy về 10^6 lần tăng mỗi thread, với OW lời gọi `is_prime(101)` giữa hai lần tăng (Hình 5, Bảng 7). Nhóm cũng đo độ công bằng như ở RQ3 và thời gian một lần tăng, tức thời gian chạy chia cho tổng số lần tăng của mọi thread.

Bằng chứng. Không counter nào mở rộng. Với OW = 0, thông lượng của `faa` giảm từ khoảng 610 triệu lần tăng mỗi giây với 1 thread xuống 42 triệu với 20 thread. Từ 8 thread, một lần tăng tốn gần như không đổi (23–30 ns). Trên `apple-m5` không thấy bước gãy rõ ở 4 thread, tức số P-core. Trên `C4M7Y670NT`, chi phí một lần tăng tăng 3,3 lần từ 6 lên 8 thread (RQ6).



Hình 6: Số lần compare-and-swap hỏng trên mỗi lần tăng thành công của các counter dùng CAS. Trục tung tuyến tính dưới 0,01 và logarit phía trên; 0 nghĩa là không lần thử nào hỏng.

Bảng 7: Trung vị thời gian chạy (s) của mọi counter trên máy `apple-m5`, `OUTSIDE_WORK` = 0 (lời gọi `is_prime(101)` giữa hai lần tăng), quy về 10^6 lần tăng mỗi thread (thời gian đo nhân với $10^6/I$; I được hiệu chỉnh riêng cho từng cấu hình). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy.

Counter \ số thread	1	2	3	4	6	8	10	12	16	20
`faa`	0,00164	0,0117	0,0238	0,0406	0,106	0,187	0,243	0,321	0,483	0,472
`faa_seqcst`	0,00166	0,0125	0,0219	0,0389	0,0911	0,163	0,299	0,343	0,448	0,557
`cas`	0,00167	0,00842	0,0219	0,0871	0,213	0,498	0,540	0,823	1,31	1,12
`cas_sleep`	0,00165	0,00341	0,00513	0,00695	0,0103	0,0138	0,0173	0,0211	0,0280	0,0351
`cas_spin`	0,00192	0,00385	0,00597	0,00772	0,0141	0,0216	0,0213	0,0271	0,0345	0,0461
`cas_yield`	0,00165	0,00340	0,00512	0,00706	0,0109	0,0162	0,0196	0,0220	0,0301	0,0360
`cas_seqcst`	0,00170	0,0177	0,0546	0,159	0,405	0,942	1,25	1,06	2,04	2,91
`lock_tas`	0,00426	0,108	0,244	0,407	0,931	1,71	2,41	3,38	4,59	6,45
`lock_tas_bo_sleep`	0,00421	0,00797	0,0121	0,0164	0,0246	0,0329	0,0456	0,0490	0,0681	0,0856
`lock_tas_bo_spin`	0,00428	0,00805	0,0122	0,0169	0,0450	0,0819	0,128	0,166	0,197	0,423
`lock_tas_bo_yield`	0,00408	0,0671	0,169	0,272	0,567	1,21	1,37	2,25	2,63	3,13
`lock_ttas`	0,00443	0,163	0,281	0,509	1,22	1,22	1,95	2,77	4,01	5,99
`lock_ttas_bo_sleep`	0,00408	0,00816	0,0122	0,0169	0,0247	0,0329	0,0407	0,0490	0,0665	0,0878
`lock_ttas_bo_spin`	0,00408	0,00802	0,0128	0,0179	0,0322	0,0567	0,0778	0,0983	0,149	0,189
`lock_ttas_bo_yield`	0,00399	0,0233	0,0955	0,0656	0,123	0,118	0,289	0,178	0,288	0,473
`lock_mcs`	0,00437	0,349	0,351	0,584	1,06	1,47	4,85	4,07	3,73	3,71
`lock_std_mutex`	0,00389	0,0146	0,0428	0,0402	0,0757	0,113	0,129	0,160	0,205	0,250
`lock_pthread_mutex`	0,00351	0,0140	0,0444	0,0371	0,0728	0,0993	0,120	0,140	0,189	0,233
`lock_os_unfair_lock`	0,00237	0,0488	0,0359	0,0693	0,0497	0,0898	0,105	0,176	0,245	0,306

`cas` chậm hơn `faa` 2,0 tới 2,7 lần từ 4 thread, nhưng ở 2 thread lại tốn ít hơn 28%. Gọi f là số lần CAS hỏng trên mỗi lần tăng (Hình 6). Ở OW = 0, f tăng đều từ 0,09 ở 2 thread lên 1,59 ở 8 thread, rồi dao động 1,4–2,2. Ở OW = 1, f đã là 0,92 ở 2 thread và ở mức 5,4–6,0 từ 8 thread. Ở OW = 8, f chỉ 0,02 ở 2 thread, tăng dần tới 1,4 ở 10 thread, rồi ở mức 1,7–2,4 khi vượt số core.

Ở OW = 0, ba biến thể CAS có backoff hỏng ít hơn `cas` gần một tới ba bậc ($f \le 0{,}02$). Khi có việc giữa hai lần tăng, `cas_yield` hỏng nhiều hơn hẳn. Ở OW = 1, f của nó khoảng 1 ở mọi số thread (0,78–1,31). Ở 4 thread, con số này tăng khoảng 50 lần so với OW = 0 (từ 0,022 lên 1,17). Ở 2 thread nó gần bằng `cas` (0,78 so với 0,92), nhưng từ 8 thread `cas` hỏng nhiều hơn 4 tới 6 lần. `cas_spin` và `cas_sleep` vẫn hỏng dưới 0,09 lần mỗi lần tăng ở OW = 1.

Khi OW $\le$ 1, counter dùng spinlock không backoff chậm nhất ở hầu hết số thread (ngoại lệ OW = 1, 3 thread, nơi mutex thư viện chậm hơn). Ở 20 thread và OW = 0, `lock_tas` chậm hơn `faa` 13,7 lần. `lock_mcs` chậm hơn `lock_tas` 3,2 lần ngay ở 2 thread. Ở OW = 0, một lần tăng của `lock_mcs` tốn 184 ns ở 8 thread và 485 ns ở 10 thread, bằng số core.

Ở OW = 0, counter dùng `lock_pthread_mutex` và `lock_std_mutex` nhanh hơn `faa` từ 4 thread (ở 4 thread `lock_std_mutex` ngang `faa`, 10,1 so với 10,2 ns). Ở 8 thread, một lần tăng tốn 12,4

và 14,1 ns, so với 23,4 ns của `faa`. Độ công bằng của hai mutex ở mọi số thread là 0,90–0,99. Ở 3 thread, chúng chậm hơn ở 4 thread (14,3–14,8 so với 9,3–10,1 ns). `lock_os_unfair_lock` cũng gần `faa` nhưng không đơn điệu (2 thread chậm hơn 3 thread, 24,4 so với 12,0 ns). Ở OW = 1, từ 8 thread mutex chậm hơn `faa` 3,0–3,4 lần.

Với OW = 1, `faa` tốn khoảng 52 ns mỗi lần tăng từ 6 tới 20 thread, gần bằng thời gian một vòng lặp của 1 thread (51,6 ns). Ở 2–4 thread nó tốn 39–44 ns. Với OW = 8, mọi counter gần nhau tới 4 thread. Counter dùng mutex chậm hơn `faa` 3,4–3,7 lần ở 6 thread và 4,8–4,9 lần ở 20 thread. `cas` chậm hơn `faa` 1,2–1,5 lần từ 10 thread.

Vì sao. Mỗi lần fetch-and-add phải giữ cache line của counter ở trạng thái độc quyền, nên các lần tăng của mọi thread diễn ra lần lượt trên một line, và thêm thread không tăng thông lượng. Từ 10 thread, chỉ 10 thread chạy cùng lúc, nên số core tranh line không tăng thêm và chi phí một lần tăng thôi tăng.

Một lần CAS hỏng đã tốn một lần kéo cache line về mà không tăng được counter. Lệnh `ldadd` của fetch-and-add thì đọc và ghi trong một bước. Nếu mỗi lần hỏng tốn một lần chuyển, thời gian của `cas` xấp xỉ thời gian của `faa` nhân $1 + f$. Mô hình bỏ qua lệnh `isb` (khoảng 12 ns) mà `NoBackoff::pause()` chờ sau mỗi lần hỏng (`backoff.hpp`). Từ 6 tới 20 thread, mô hình lệch tỉ số đo được không quá 18% (16 thread: đo 2,71, mô hình 3,18). Ở 4 thread, mô hình chỉ giải thích 1,52 trong 2,14 lần. Số lần hỏng thôi tăng từ 10 thread, vì chỉ 10 thread tranh line cùng lúc.

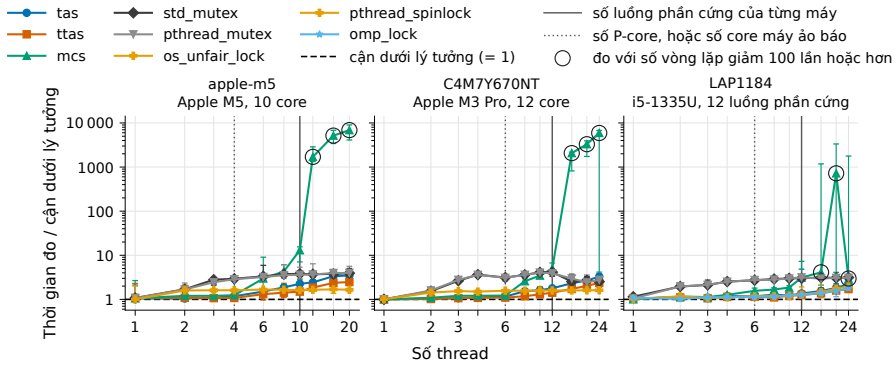
Ở 2 thread, chỉ có 0,09 lần CAS hỏng trên mỗi lần tăng, nên gần như mọi lần tăng thành công ngay lần đầu. Mô hình $1 + f$ không giải thích được vì sao `cas` lại rẻ hơn `faa` ở đó. Kiến trúc ARMv8.1 cho phép phần cứng thực hiện `ldadd` ở gần cache dùng chung thay vì trong L1 của core gọi. Nếu M5 làm vậy, mỗi lần `ldadd` tốn một lượt đi về tới cache dùng chung. Một thread chạy vòng CAS thì có thể tăng vài lần liên tiếp trên line đang nằm sẵn trong L1 của nó. Đây là giả thuyết. RQ6 cho thêm bằng chứng gián tiếp từ hai máy khác. Ghi lại giá trị trả về của từng lần tăng sẽ cho biết mỗi thread tăng liên tiếp bao nhiêu lần trước khi line rời core của nó, và cho phép kiểm giả thuyết này.

Khi có `is_prime` xen giữa hai lần tăng, nhóm cho rằng line đã sang core khác trước lần đọc kế tiếp. Giữa lần đọc và lần CAS vì vậy dễ có thread khác chen vào, phù hợp với $f = 0{,}92$ ở 2 thread và OW = 1. Ở OW = 8, các lần tăng thưa (khoảng 400 ns `is_prime` giữa hai lần), nên ít thread chen vào hơn. `cas_yield` cho thấy cùng hiệu ứng. Ở OW = 0, nó chờ 64 lần đầu bằng đúng lệnh `isb` như `cas`, nên nhóm chưa giải thích được vì sao khi đó nó hỏng ít hơn. Độ công bằng thấp của nó (0,10–0,39 từ 4 thread) cho thấy nó dồn lần tăng về ít thread. Khi có việc xen giữa, các lần tăng liên tiếp của một thread không còn xảy ra, và f của nó lên khoảng 1. `cas_sleep` và `cas_spin` hỏng ít vì cho các thread tăng lần lượt (độ công bằng 0,05 và 0,08 ở 20 thread, OW = 0).

Mức 52 ns của `faa` ở OW = 1 không đổi từ 6 tới 20 thread. Mỗi thread vì vậy mất khoảng $p \times 52$ ns cho một vòng lặp (khoảng 1 µs ở 20 thread), trong khi `is_prime(101)` chỉ tốn khoảng 50 ns. Vậy các lần tăng diễn ra lần lượt, mỗi lần khoảng 52 ns. Ở OW = 0, cùng counter chỉ tốn 23,6 ns mỗi lần ở 20 thread. Nhóm chưa giải thích được vì sao một lần tăng đắt gấp đôi khi có `is_prime` xen giữa. Một khả năng là ở OW = 0 phần cứng xử lý liên tiếp nhiều lần tăng của cùng một core, còn ở OW = 1 thì không. Khả năng này chưa được kiểm. Mức 52 ns gần bằng thời gian một vòng của 1 thread chỉ là trùng hợp. Trên `LAP1184`, `faa` ở OW = 1 cũng dừng ở 50–57 ns từ 6 thread, dù 1 thread tốn 174 ns. Ở 2–4 thread, các thread có thể cùng nằm trong cụm 4 P-core dùng chung L2, nên một lần tăng rẻ hơn (39–44 ns). Đây cũng là giả thuyết.

Thời gian một lần tăng của `cas_sleep` bằng giá trị 1 thread trong khoảng 11% ở mọi số thread và mọi OW. Độ công bằng của nó xấp xỉ $1/p$, đúng như khi các thread chạy lần lượt. Vì vậy, ở 20 thread, `cas_sleep` nhanh hơn `cas` 32 lần khi OW = 0 nhưng chậm hơn 3,1 lần khi OW = 8. Trong trường hợp sau, chạy tuần tự bỏ phí phần `is_prime` lẽ ra chạy song song.

Trong counter dùng lock, lock và giá trị nằm trên hai cache line khác nhau (`counter.hpp`). Mỗi lần tăng vì vậy chuyển ít nhất hai line, và các thread chờ còn tranh line của lock. Điều này phù hợp với việc spinlock không backoff chậm nhất. `lock_mcs` trao lock theo thứ tự đến, nên mỗi



Hình 7: So sánh các máy, phần lock: thời gian đo chia cho cận dưới lý tưởng của chính máy đó, `OUTSIDE_WORK` = 0, cùng thang đo. Chia cho cận dưới khử khác biệt về tốc độ core.

lần tăng chuyển cả lock lẫn giá trị sang core khác, phù hợp với việc nó chậm hơn `lock_tas` ngay ở 2 thread. Bước nhảy của `lock_mcs` ở 10 thread phù hợp với cơ chế MCS gãy mạnh tại số core ở RQ2. Khi thread kế tiếp trong hàng mất core, cả hàng phải chờ nó.

Ở OW = 0 và từ 4 thread, một lần tăng dưới mutex rẻ hơn một lần tăng của `faa`. Nhóm cho rằng người giữ lock thực hiện nhiều lần tăng liên tiếp trên line đã nằm trong cache của mình, còn các thread chờ ngủ trong kernel và không tranh line. Với `faa`, mọi thread đang chạy cùng gửi lần tăng tới counter. Nhóm chưa đo độ dài các chuỗi này. Độ công bằng 0,90–0,99 cho thấy nếu có các chuỗi như vậy thì chúng ngắn so với cả lần chạy. Với `lock_os_unfair_lock`, nhóm cho rằng mức dồn lần tăng phụ thuộc vào lúc thread chờ được đánh thức, nhưng chưa kiểm. Việc hai mutex chậm hơn ở 3 thread so với 4 thread thì nhóm chưa giải thích được. RQ6 thấy hiện tượng tương tự trên `C4M7Y670NT` ở 3–4 thread.

Khi có việc giữa hai lần tăng, người nhả lock đi tính `is_prime` thay vì lấy lại lock ngay, nên lần lấy kế tiếp thường thuộc về một thread đang ngủ và phải đánh thức nó. Vì vậy ở OW = 1 và 8, counter dùng mutex chậm hơn `faa` vài lần. Ở OW = 8, tới 4 thread `is_prime` chiếm phần lớn thời gian nên mọi counter gần nhau. `cas` chỉ tách khỏi `faa` từ 10 thread, khi f vượt 1.

Thứ tự `seq_cst` làm `faa` dao động trong khoảng ±23%, không theo hướng nào, nhưng làm `cas` chậm khoảng 2 lần ở OW = 0 và gần gấp đôi số lần hỏng. Ở OW ≥ 1, khác biệt biến mất. Nhóm giả thuyết rằng lệnh `casal` (acquire và release) ngăn các lần lặp chồng lên nhau. Lần đọc đầu của `cas_seqcst` vẫn là `load(relaxed)`, nên không có `ldar` (`counter.hpp`).

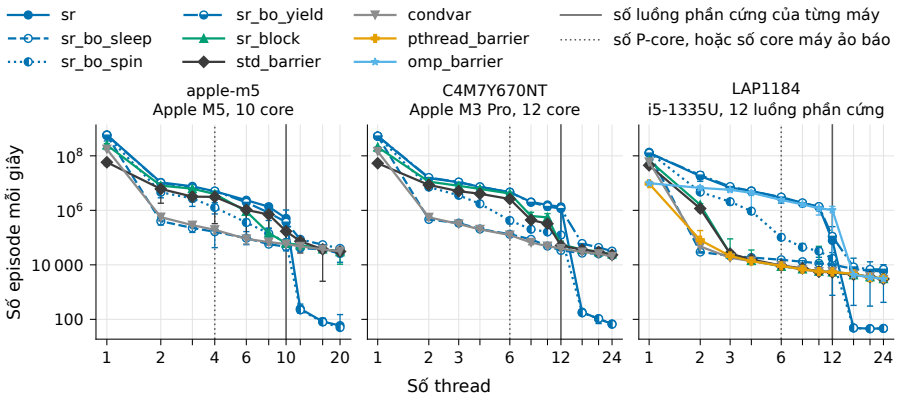
Độ vững. Nhóm không phân tích `lock_mcs` trên 10 thread, vì các ô đó đo với rất ít vòng lặp (mục Các yếu tố đe dọa độ tin cậy). Chi phí `seq_cst` của CAS không xuất hiện nhất quán trên `C4M7Y670NT` (RQ6), nên kết luận về nó chỉ dựa trên `apple-m5`.

6.4 So sánh giữa các máy

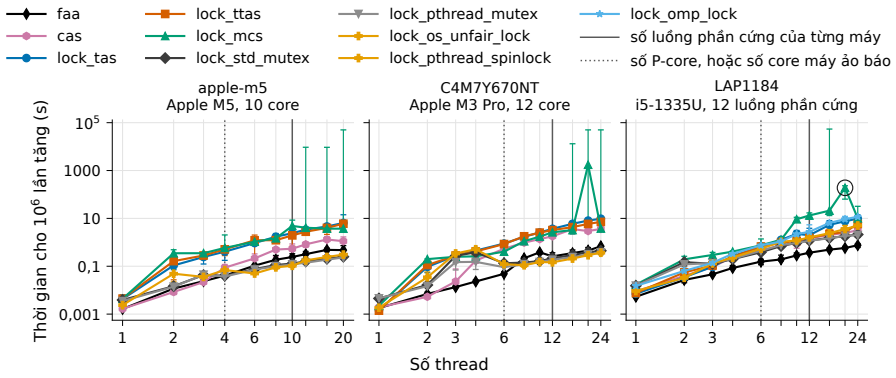
6.4.1 RQ6: Kết luận trên máy khác

Trả lời RQ6. Một phần. Thứ hạng chính giữ trên cả ba máy. TTAS nhanh hơn TAS, fetch-and-add nhanh hơn vòng CAS từ 4 thread, `sr_bo_yield` tốt nhất khi quá tải, và backoff bằng ngủ tuần tự hoá các thread. Giữa hai máy Apple, điểm gãy dời theo số P-core và điểm sụp dời theo số core. Trên `LAP1184` (x86-64, Linux trong máy ảo WSL2), `seq_cst` không còn tốn thêm gì, barrier spin chậm 17 lần ngay ở 12 thread (bằng số luồng phần cứng), `sleep_for` với yêu cầu 100 ns ngủ trung vị khoảng 62 µs, và MCS quá tải cho hai cực thời gian.

`C4M7Y670NT` là Apple M3 Pro với 6 P-core và 6 E-core, không SMT. Hai máy Apple cùng chạy



Hình 8: So sánh các máy, phần barrier: số episode mỗi giây theo số thread, cùng thang đo.



Hình 9: So sánh các máy, phần counter ở mức tranh chấp cao nhất (`OUTSIDE_WORK` nhỏ nhất của từng máy): thời gian chạy quy về 10^6 lần tăng mỗi thread, cùng thang đo.

macOS (27.0.1 trên `apple-m5`, 26.7.1 trên `C4M7Y670NT`) và Apple clang 21, nhưng khác thế hệ chip (M5, M3 Pro), cách chia cache L2 (16 MiB cho 4 P-core so với 16 MiB cho 6 P-core) và tải nền. Khác biệt cấu trúc lớn nhất là số P-core và số core, nên hai máy này cho phép so tác động của hai con số đó. `LAP1184` khác `apple-m5` ở kiến trúc, memory model, compiler, hệ điều hành, và chạy trong máy ảo (mục Các yếu tố đe dọa độ tin cậy). Cơ chế nào chưa có phép đo riêng được ghi là giả thuyết. Hình phần lock (Hình 7) chia thời gian cho cận dưới của chính máy đó, nên khử được khác biệt tốc độ core. Hình barrier và counter (Hình 8 và 9) vẽ số đo thô, nên chỉ so được hình dạng đường cong. Ô `apple-m5` đã được phân tích ở RQ2 tới RQ5. Trên ô `LAP1184`, vạch chấm ở 6 là số core máy ảo báo (6 core vật lý, mỗi core 2 luồng), còn vạch liền ở 12 là số luồng phần cứng. CPU thật là i5-1335U với 2 P-core có Hyper-Threading và 8 E-core, tức 10 core và 12 luồng.

Bằng chứng. Lock (Hình 7; Bảng 16 và 21). Trên `C4M7Y670NT`, TAS chậm 1,15 lần cận dưới ở 6 thread và 1,53 ở 8 thread, MCS 1,23 rồi 2,58. Trên `apple-m5` chỗ gãy này nằm ở 4 lên 6 thread. MCS chậm 4,99 lần ở 12 thread (bằng số core) và 2 070–5 920 lần khi vượt. Hai mutex thư viện chậm 3,9–4,2 lần cận dưới ở 12 thread nhưng chỉ 2,4–3,0 lần từ 16 thread. Trên `LAP1184`, tới 12 thread TAS, TTAS, `pthread_spinlock` và `omp_lock` chạy trong 1,36 lần cận dưới, không gãy ở 6 hay 10, và ở 24 thread ở mức 1,7–2,2. MCS chậm 1,57–1,67 lần ở 6–8 thread, 1,85 lần ở 10 thread và 3,13 lần ở 12 thread. TAS chậm hơn TTAS 1,14 và 1,30 lần ở 20 và 24 thread (`apple-m5` 1,40 và `C4M7Y670NT` 1,32 ở 20 thread). Mutex thư viện chậm 2,0 lần cận dưới ngay ở 2 thread rồi tăng chậm tới khoảng 3. Các điểm khoanh tròn khi quá tải được bàn ở mục Độ vững.

Barrier (Hình 8; Bảng 17 và 22). Trên `C4M7Y670NT`, mỗi thread thêm vào làm một episode của `sr` dài thêm 36–39 ns tới 6 thread, rồi 139 ns từ 6 lên 8 thread. `sr_block` và `std_barrier` tụt 6,6 và 5,9 lần ở cùng chỗ, còn trên `apple-m5` chỗ tụt nằm ở 4 lên 6. Từ 10 lên 12 thread, bằng số core, hai barrier này tụt thêm 9,9 và 6,3 lần. `sr` vẫn đạt 1,35 triệu episode mỗi giây ở 12 thread và chỉ sụp khi vượt số core (184 ở 16 thread). `condvar` đạt 556 và 200 nghìn episode mỗi giây ở 2 và 4 thread, so với 565 và 193 nghìn trên `apple-m5`. Trên `LAP1184`, `sr` ở 1 thread chỉ đạt 128 triệu (`apple-m5` 589 triệu), nên mọi đường cong nằm thấp hơn. Từ 4 tới 10 thread, `sr` dài thêm 77–101 ns mỗi thread, không gãy ở 6 hay 10. Ở 12 thread nó chậm 17 lần so với 10 thread. Từ 16 thread, `sr` và `sr_bo_spin` chỉ còn 45–49 episode mỗi giây. `omp_barrier` chậm hơn `sr` từ 1 tới 10 thread (9,92 so với 128 triệu ở 1 thread, 1,19 so với 1,35 triệu ở 10 thread). Nó vượt `sr` ở 12 thread (1,03 triệu), rồi sụp còn 4,25 nghìn ở 16 thread, ngang các barrier chờ chặn (4,30–4,67 nghìn). Bốn barrier chờ chặn rơi xuống 18–26 nghìn ngay ở 3 thread. Ở 2 thread, `sr_block` và `std_barrier` vẫn đạt 1,60 và 1,16 triệu, còn `condvar` và `pthread_barrier` chỉ đạt 47 và 81 nghìn. `sr_bo_spin` chậm hơn `sr` 5,5 và 28 lần ở 4 và 6 thread (`apple-m5` 3,9 và 6,6). Ở 2 thread, `sr_bo_sleep` chậm hơn `sr` 611 lần (`apple-m5` 26). Khi quá tải, `sr_bo_yield` nhanh nhất trên cả ba máy.

Counter (Hình 9; Bảng 18 và 23). Trên `C4M7Y670NT`, một lần tăng của `faa` tốn 5,8–8,1 ns ở 4–6 thread và 21–38 ns từ 8 thread. `cas` nhanh hơn `faa` ở 2 thread (0,76 lần, `apple-m5` 0,72), rồi chậm hơn 3,6–11 lần từ 4 thread (43–90 ns ở 4–6 thread). Counter dùng mutex nhanh hơn `faa` từ 8 thread (`lock_std_mutex` 17,8 so với 27,0 ns), nhưng ở 3–4 thread ba lock thư viện chậm hơn `faa` 6,6 tới 25 lần. Trên `LAP1184`, `cas` chậm hơn `faa` ngay từ 2 thread (1,49 lần) và 2,9 tới 4,3 lần từ 4 thread, dù số lần hỏng ở 8 thread gần bằng `apple-m5` (1,62 và 1,59). Counter dùng mutex chậm hơn `faa` 2,3 tới 4,0 lần từ 3 thread. `lock_mcs` tăng từ 159 lên 918 ns mỗi lần tăng khi đi từ 8 lên 10 thread, ở cả bảy lần chạy. Từ 12 thread, `lock_omp_lock` chậm hơn `lock_tas` 1,2–1,4 lần, nên trên Hình 9 nó chỉ nhanh hơn `lock_mcs`.

Vì sao: hai máy Apple. Các điểm gãy dời theo đúng số P-core và số core. Chỗ gãy của spinlock, của độ dốc barrier spin và của các barrier quay-rồi-ngủ nằm ngay sau số P-core. Điều này phù hợp với giải thích ở RQ2 và RQ4. Thread vượt số P-core chạy trên E-core chậm hơn, và cache line phải đi giữa hai cụm core. `faa` trên `C4M7Y670NT` phù hợp với phần sau. Trong cụm 6 P-core dùng chung L2 nó rất rẻ, và tốn gấp 2,7 tới 4,7 lần khi có thread ở cụm E-core. Ở 4–6 thread, `cas` tốn gấp 7,4–11 lần `faa`, lớn hơn khoảng 3 lần mức mô hình $1 + f$ của RQ5 (2,4 và 3,8). Mô hình đó không áp được cho máy này. Khoảng chênh lớn phù hợp với giả thuyết của RQ5 rằng `ldadd` được thực hiện ở cache dùng chung, còn CAS phải kéo line về L1. Vì `faa` rẻ như vậy trong cụm P-core, counter dùng mutex chỉ thắng nó từ 8 thread, khi line phải đi giữa hai cụm. MCS và barrier spin

sụp khi vượt số core, vì thread chờ phải đợi một lượt lập lịch (RQ2, RQ4). Các chi phí do hệ điều hành quyết định thì bằng nhau. `sleep_for` với yêu cầu 100 ns ngủ trung vị 2,67 µs trên cả hai máy, và `condvar` gần như trùng.

Trên `C4M7Y670NT`, độ chậm của hai mutex thư viện giảm khi vượt số core. Cùng lúc, độ công bằng của `std_mutex` giảm từ 0,93 ở 12 thread xuống 0,68–0,76 từ 16 thread. Điều này phù hợp với việc người vừa nhả lock lấy lại nó trước khi thread được đánh thức có core, nên ít lần trao lock phải qua kernel hơn. Trên `apple-m5`, độ chậm của mutex vẫn ở 3,6–4,0 lần khi vượt số core, nên nhóm chưa giải thích được khác biệt giữa hai máy. Bước tụt thứ hai của `sr_block` và `std_barrier` ở 12 thread phù hợp với cơ chế của RQ4. Nhóm cho rằng khi mọi core đều bận, thread chờ dễ mất core, nên nhiều episode dài quá 4 µs hơn, và thread chờ ngủ rồi phải được đánh thức qua kernel. Nhóm chưa đo tần suất các lần ngủ này.

Ở số thread bằng số core, `sr` trên `apple-m5` chậm 2,7 lần (8 lên 10 thread), còn trên `C4M7Y670NT` chỉ 1,17 lần (10 lên 12 thread). Điều này phù hợp với tải nền cao hơn của `apple-m5` trong đợt đo (trung vị load average 1 phút trong phần barrier là 26 so với 18). Với MCS, phép đo lại khi tải thấp cho điểm gãy tại số core nhỏ hơn (RQ2). Từ 12 thread, `cas` và `cas_seqcst` trên `C4M7Y670NT` cùng chia hai cực (khoảng 137–157 và 206–279 ns mỗi lần tăng). Tỉ số trung vị của chúng (0,69 tới 1,68) chỉ phản ánh trung vị rơi vào cực nào. Chi phí `seq_cst` của CAS ở RQ5 vì vậy chỉ được `apple-m5` ủng hộ. Thời gian tăng bất thường của counter dùng mutex ở 3–4 thread trên `C4M7Y670NT` (và ở 3 thread trên `apple-m5`, RQ5) nhóm chưa giải thích được. Các ô này phân tán mạnh (`lock_std_mutex` ở 4 thread, 38–108 ns).

Vì sao: `LAP1184`. Thứ nhất, critical section dài hơn. Một lời gọi `is_prime(1009)` tốn 1 640 ns, gấp 3,2 lần `apple-m5`. Khoảng trống mỗi lần trao lock của TAS ở 20 thread được ước lượng bằng thời gian mỗi lần lấy lock trừ thời gian một vòng lặp của 1 thread. Nó gần bằng nhau trên hai máy (1,34 và 1,27 µs). Chia cho critical section dài hơn, nó cho độ chậm nhỏ hơn. Với mutex, khoảng trống là 3,4–3,5 µs, gấp 2,2–2,3 lần `apple-m5`. Điều này phù hợp với việc đánh thức qua futex (lời gọi hệ thống của Linux để thread ngủ và được đánh thức trên một biến) trong máy ảo đắt hơn. Đường spinlock không gãy ở một số thread cố định. Nhóm cho rằng Windows đặt 12 CPU ảo lên P-core và E-core mà Linux không biết, nên chênh lệch tốc độ core trải đều trên mọi số thread.

Thứ hai, x86-64 và TSO. Mọi phép đọc-sửa-ghi atomic là lệnh có tiền tố `lock`, vốn đã là một rào chắn đầy đủ, nên `seq_cst` không đổi lệnh máy. `cas_seqcst` bằng 0,75 tới 1,14 lần `cas` và `faa_seqcst` bằng 0,90 tới 1,13 lần `faa`. `lock xadd` và `lock cmpxchg` đều kéo line về L1 của core gọi, nên không có hiệu ứng `ldadd` ở xa. Đây là giả thuyết, phù hợp với việc `cas` không thắng `faa` ở 2 thread. Lệnh `pause` tốn trung vị 51 ns, gấp 4 lần `isb`. Mỗi lần CAS hỏng chờ một lệnh như vậy, lâu hơn một lần `faa` (24–32 ns từ 8 thread). Điều này phù hợp với tỉ số `cas`/`faa` lớn hơn khi số lần hỏng như nhau. `sr_bo_spin` chờ theo đơn vị `pause`, nên cùng số lần chờ kéo dài gấp 4 lần, phù hợp với việc nó tụt sớm hơn. Nhóm cho rằng `pause` cũng làm thưa vòng `exchange` của TAS, nên khoảng cách TAS–TTAS nhỏ hơn trên hai máy Apple.

Thứ ba, `sleep_for` trong máy ảo. Với yêu cầu 100 ns, `sleep_for` ngủ trung vị 61,7 µs, gấp 23 lần hai máy Apple (2,67 µs). Với mọi yêu cầu từ 1 ns tới 1 µs, trung vị đều nằm trong 61–63 µs. Nhóm cho rằng phần lớn khoảng này là timer slack mặc định 50 µs của Linux (khoảng trễ kernel được phép thêm khi đánh thức để gộp các lần hẹn giờ) cộng độ trễ bộ hẹn giờ của máy ảo. Một lần ngủ dài bằng khoảng 37 critical section, so với khoảng 5 trên `apple-m5`. Vì vậy `sr_bo_sleep` chậm 611 lần `sr`. Khi có việc ngoài critical section (OW = 3), `tas_bo_sleep` chậm hơn TAS 1,5–2,0 lần ở 3–16 thread. Trên `apple-m5`, nó chậm hơn TAS ở 3–6 thread (1,33 lần ở 6 thread) và nhanh hơn từ 8 thread (0,69 lần ở 20 thread). Độ công bằng của `tas_bo_sleep` (0,06 ở 24 thread) cho thấy các thread chạy lần lượt.

Thứ tư, đánh thức qua futex. Barrier chờ chặn tốn 39–74 µs mỗi episode ở 3–4 thread, gấp 11–16 lần `condvar` trên `apple-m5` (3,5–5,2 µs). Nhóm cho rằng `atomic::wait` của libstdc++ chỉ quay rất ngắn trước khi gọi futex, nên `sr_block` và `std_barrier` ngủ ngay từ 3 thread. Ở 2 thread, thread tới sau thường tới khi thread kia còn quay, nên hai barrier này chưa phải ngủ. Từ 3 thread, thread tới đầu phải chờ hai thread khác, lâu hơn khoảng quay đó. Đây là giả thuyết. Với

counter dùng mutex, nhóm cho rằng glibc gọi `futex_wake`, một syscall, ở mỗi lần nhả lock khi đã có thread chờ, nên chuỗi lần tăng liên tiếp của người giữ lock (giả thuyết ở RQ5) không còn rẻ. `lock_omp_lock` gần `lock_tas` tới 10 thread, phù hợp với việc libgomp quay trên lock một thời gian trước khi ngủ bằng futex (theo tài liệu `GOMP_SPINCOUNT`). Nhóm cho rằng từ 12 thread, lượt quay thường hết trước khi lock rảnh, nên thread ngủ và phải được đánh thức qua futex. Nhóm chưa kiểm cơ chế này.

Thứ năm, mọi CPU ảo cùng bận. `C4M7Y670NT` cũng có 12 luồng phần cứng nhưng `sr` không chậm rõ ở 12 thread, nên điểm gãy ở 12 thread là đặc thù của `LAP1184`. `omp_barrier` cũng quay ở 12 thread mà vẫn đạt khoảng 1 µs mỗi episode, nên riêng việc 12 thread cùng quay không làm barrier chậm. Nhóm cho rằng điểm khác là ai nhường CPU. Khi một thread của `sr` mất CPU, vì một tiến trình khác trong máy ảo hay vì Windows lấy CPU ảo, 11 thread còn lại vẫn quay và giữ CPU, nên cả barrier chờ tới khi thread đó chạy lại. libgomp quay một thời gian rồi ngủ bằng futex (theo tài liệu `GOMP_SPINCOUNT`), nên trả CPU lại. `sr_bo_yield` cũng nhường CPU và đạt 111 nghìn episode mỗi giây ở 12 thread, hơn `sr` (78,2 nghìn). Pause-loop exiting, tức cơ chế cho hypervisor lấy lại CPU ảo đang quay lâu bằng lệnh `pause`, là một giải thích khác (mục Các yếu tố đe dọa độ tin cậy). Ở 1 thread, `omp_barrier` chỉ đạt 9,92 triệu episode mỗi giây, nên mỗi episode của libgomp có chi phí cố định lớn hơn `sr`. Vì vậy nó chậm hơn `sr` khi mọi thread có CPU, và khoảng cách thu hẹp khi số thread tăng. Từ 16 thread, nó ngang các barrier chờ chặn. Điều này phù hợp với việc khi đó thread của nó hết lượt quay và ngủ bằng futex ở hầu hết episode. MCS chậm 3,13 lần cận dưới ở 12 thread (1,85 lần ở 10 thread), phù hợp với cùng cơ chế. Thread kế tiếp trong hàng đợi mất CPU thì cả hàng đợi chờ (RQ2). Ở counter, `lock_mcs` gãy từ 10 thread. Vì sao điểm này nằm ở 10 chứ không ở 12 thì nhóm chưa kiểm được.

Độ vững. Khi quá tải, MCS cho hai cực thời gian. Trên hai máy Apple, gần như mọi lần chạy quá tải đều chậm (41 trên 42 lần). Trên `LAP1184`, trong 21 lần chạy ở 16, 20 và 24 thread, mỗi lần chỉ có khoảng 1,5 ms công việc. Có 14 lần xong trong 2,7–7,1 ms và 7 lần mất 1,06–4,92 s, trong đó cả hàng đợi dừng cùng nhau. Trung vị rơi vào cực nhanh ở 16 và 24 thread, cực chậm ở 20 thread, rồi được nhân lên 1 600–2 400 lần khi quy đổi. Vì vậy điểm thấp ở 24 thread trên Hình 7 không có nghĩa MCS hồi phục (mục Các yếu tố đe dọa độ tin cậy). Ở counter, trung vị của `lock_mcs` ở 20 thread trên `C4M7Y670NT` rơi vào một lần chạy mà cả hàng đợi dừng, nên gai ở đó không đo chi phí counter. Số vòng lặp của các ô `lock_mcs` quá tải nêu ở mục Các yếu tố đe dọa độ tin cậy. Các khác biệt khác giữa các máy, như barrier chờ chặn và sleep backoff, lớn hơn nhiều lần khoảng dao động giữa các lần lặp.

7 Các yếu tố đe dọa độ tin cậy

Construct validity. Cận dưới của phần lock dùng t đo ở 1 thread, nên chứa chi phí lấy và nhả lock của lock nhanh nhất. Ở $N = 11$, chi phí này chênh 41 % giữa các lock (Bảng 12, phụ lục). Cận dưới bỏ qua giới hạn 10 core khi $p > 10$ và $\mathrm{OW} = 15$ (ở 20 thread sàn thật cao hơn 1,6 lần). Cận dưới còn giả định mọi core chạy `is_prime` nhanh như ở lần chạy 1 thread. Khi số thread vượt số P-core, một phần việc chạy trên E-core chậm hơn, nên một phần độ chậm không do lock. Ở 6 thread và $\mathrm{OW} = 15$, khi lock chưa bận liên tục, mọi lock đã chậm 1,53–2,03 lần cận dưới. Phép quy về cùng số vòng lặp giả định thời gian tỉ lệ với số vòng. Giả định này yếu nhất ở MCS quá tải (18–286 vòng mỗi thread trên `apple-m5`, 15–195 trên `C4M7Y670NT`, 10–56 trên `LAP1184`), ở counter `lock_mcs` quá tải (10–200 lần tăng mỗi thread trên cả ba máy) và ở các ô barrier sụp (50–69 episode mỗi lần chạy). Trên `LAP1184`, ở $N = 1009$ và $\mathrm{OW} = 0$, phần lớn lần chạy MCS ở 16 và 24 thread xong trong vài mili giây, còn số ít mất 1,2–2,6 s. Trung vị của hai ô này rơi vào nhóm nhanh, nên chỉ là cận dưới của chi phí MCS. Vài ô MCS quá tải chỉ còn 4–6 giá trị sau khi vượt thời hạn (mục Phương pháp đo). Ở $\mathrm{OW} = 8$, thời gian của mọi counter, kể cả `faa`, tăng ít nhất gấp đôi từ 4 lên 10 thread, và nhóm không có phép đo đối chứng để tách phần do E-core. Phép đo khoảng trống trao lock thêm hai lần đọc đồng hồ vào mỗi critical section, nên báo cáo chỉ dùng thống kê khoảng trống của nó, không dùng thời gian chạy. Độ công bằng so thời điểm

kết thúc của các thread, nên không thấy các đợt độc chiếm luân phiên ngắn.

Internal validity. Không lần chạy nào ghim thread vào core. `apple-m5` có 4 P-core và 6 E-core, và macOS không cho ghim thread. Ở $N = 1009$, thời gian một lời gọi `is_prime` trong 420 lần chạy 1 thread đi từ 468 tới 1 713 ns. Nhóm cho rằng độ phân tán này phần lớn do thread rơi vào E-core, nhưng không tách được ảnh hưởng của E-core với tải nền. Load average 1 phút trong đợt đo lock có trung vị khoảng 25 ở mọi số thread, kể cả lần chạy 1 thread (12,4–51,2). Con số này cao gấp đôi số thread trung bình theo thời gian của đợt đo lock (12,0). Nhóm đo lại năm lock (`tas`, `ttas`, `mcs`, `std_mutex`, `os_unfair_lock`) ở $N = 1009$, OW $= 0$, 1–10 thread, 7 lần lặp, mỗi lần chạy chỉ bắt đầu khi load average dưới 3 hoặc sau 2 phút chờ (trung vị 2,9). Ở cấu hình này, các đường cong giữ hình dạng khi tải thấp, còn điểm gãy của MCS ở 10 thread nhỏ đi một nửa (0,53 lần, tức chậm khoảng 7 lần cận dưới thay vì 13). Ngoài MCS ở 10 thread, trung vị mới bằng 0,83–1,05 lần trung vị của đợt đo chính, tức lệch tới 17 %. Báo cáo không diễn giải những khác biệt nhỏ hơn mức này. Lần đo lại khác đợt đo chính cả về load average lẫn phiên chạy và khoảng nghỉ giữa các lần chạy, nên không tách được riêng ảnh hưởng của tải. Vùng vượt số core chưa được đo lại. Nhóm không đọc nhiệt độ hay tần số CPU. Thời gian 1 thread không trôi quá 3,4 % qua 7 lượt lặp. Với 7 lần lặp, nhóm không tính khoảng tin cậy. Trung vị không thể hiện phân bố hai đỉnh, và chỉ thanh min–max cho thấy phần này. Trên `LAP1184`, load average (5,90–19,6) chỉ đếm tiến trình trong máy ảo. Tải của Windows, việc Windows xếp lịch cho các CPU ảo, nguồn điện và chế độ điện trong lúc đo không được ghi lại. Linux không biết CPU ảo nào nằm trên P-core hay E-core, và `lscpu` báo tần số cố định 2 496 MHz, nên không thấy được throttling. Thời gian một lời gọi ở 1 thread, $N = 1009$, đi từ 1 463 tới 3 176 ns qua 455 lần chạy. `sleep_for` với yêu cầu 100 ns ngủ trung vị khoảng 62 µs. Trong con số này, nhóm không tách được phần của bộ hẹn giờ ảo của Hyper-V với phần của timer slack, tức khoảng trễ Linux được phép thêm vào khi đánh thức để gộp các lần hẹn giờ. Pause-loop exiting, cơ chế hypervisor tạm dừng một CPU ảo quay vòng chờ quá lâu, và việc đánh thức thread qua ngắt ảo là những giải thích khác, chưa đo, cho việc barrier spin chậm ngay ở 12 thread.

External validity. Số đo đến từ ba máy, gồm hai máy Apple Silicon (Apple clang 21, macOS) và một máy x86-64 (g++ 11.5, Linux trong máy ảo WSL2), cả ba đều `-O2`. Hai máy Apple khác nhau ở số P-core và số core, nhưng cũng ở thế hệ chip (M3 Pro, M5), dung lượng L2 mỗi cụm và phiên bản macOS. Vì vậy việc điểm gãy dời theo số P-core phù hợp với giải thích hai loại core, nhưng không chứng minh nó. Máy x86-64 khác hai máy Apple cùng lúc ở kiến trúc, memory model, compiler, hệ điều hành và việc có máy ảo hay không. Vì vậy RQ6 chỉ gán một khác biệt của máy này cho một nguyên nhân khi có bằng chứng riêng, như `seq_cst` với TSO. Nhóm không có máy x86 chạy Linux trực tiếp, máy chạy Windows trực tiếp, hay máy nhiều socket. `is_prime` không chạm dữ liệu dùng chung. Critical section thật thường ghi dữ liệu dùng chung, nên chi phí chuyển giao lock thực tế có thể lớn hơn.

8 Kết luận

Nhóm cài đặt ba spinlock, một sense-reversing barrier và ba họ concurrent counter, kiểm tra chúng bằng stress test, ThreadSanitizer và mutation testing, rồi đo chúng trên ba máy. RQ2 tới RQ5 dựa trên `apple-m5`. RQ1 và RQ6 có số liệu của cả ba máy.

RQ1. Không bài kiểm tra nào tìm thấy lỗi trên ba máy, và bộ kiểm tra bắt được cả 14 biến thể cố ý làm hỏng. Lỗi thuật toán lộ ra dưới stress test, còn lỗi memory order chủ yếu do ThreadSanitizer bắt.

RQ2. TTAS mở rộng tốt nhất trong ba spinlock tự viết (2,48 lần cận dưới ở 20 thread), còn lock thư viện không sụp vì thread chờ ngủ trong kernel. MCS gãy mạnh tại số core và sụp khi vượt số core, vì nó trao lock cho đúng một thread kế tiếp, kể cả khi thread đó không có core.

RQ3. Backoff giúp TAS nhiều nhất (ở 20 thread, TAS với sleep backoff chỉ tốn 0,31 lần thời gian của TAS), nhưng TTAS với sleep backoff chậm hơn TTAS 6,1–6,8 lần ở 16 và 20 thread với OW $= 3$. Lợi ích của backoff đến từ việc giữ lock ở một chủ, nên lock nhanh nhất là lock kém

công bằng nhất.

RQ4. Với barrier spin, một episode tốn thêm khoảng 53 ns cho mỗi thread trong cụm P-core, và số episode mỗi giây giảm khoảng 2 000 lần khi số thread vượt số core. Khi đó, chờ bằng `sched_yield` nhanh nhất.

RQ5. Từ 4 thread, fetch-and-add nhanh hơn vòng CAS 2,0 tới 2,7 lần, vì số lần CAS hỏng trên mỗi lần tăng tăng từ 0,09 ở 2 thread lên 1,59 ở 8 thread. Khi không có việc giữa hai lần tăng, CAS có backoff nhanh hơn fetch-and-add từ 2 thread vì backoff dồn các lần tăng về từng thread, còn `lock_std_mutex` và `lock_pthread_mutex` ngang fetch-and-add ở 4 thread và nhanh hơn từ đó trở lên.

RQ6. Một phần, vì thứ hạng chính giữ trên cả ba máy nhưng giữa hai máy Apple, điểm gãy dời theo số P-core và điểm sụp của MCS dời theo số core. Trên `LAP1184`, `seq_cst` không tốn thêm gì, barrier spin chậm 17 lần ngay ở 12 thread, và `sleep_for` với yêu cầu 100 ns ngủ trung vị khoảng 62 µs.

Điểm gãy và điểm sụp của MCS do scheduler, không do lưu lượng cache coherence. Trong một phép đo riêng ở tải thấp, chỉ 0,25 % lần trao lock của MCS ở 10 thread chờ quá 100 µs, nhưng chúng chiếm 47 % thời gian chạy. Điểm gãy tại số P-core phù hợp với giả thuyết hai loại core. Nó dời từ 4 lên 6 thread trên máy có 6 P-core, nhưng chưa có phép đo riêng cho cơ chế này. Nhóm cho rằng mutex thư viện thắng fetch-and-add vì người giữ lock tăng nhiều lần liên tiếp trên cache line của mình, nhưng chưa đo độ dài các chuỗi này. Trên `apple-m5`, `seq_cst` làm CAS chậm khoảng 2 lần. Nhóm cho rằng lệnh `casal` ngăn các lần lặp chồng lên nhau, nhưng `C4M7Y670NT` không cho thấy chi phí này nhất quán.

Khi số thread có thể vượt số core, nhóm khuyến nghị dùng lock thư viện. Trên macOS, `os_unfair_lock` nhanh nhất trong các lock thư viện, chậm 1,6–1,7 lần cận dưới khi không có việc ngoài critical section. TAS với sleep hoặc yield backoff còn nhanh hơn ở chế độ đó, nhưng độ công bằng chỉ còn $1/p$, vì một thread giữ lock tới hết vòng lặp của nó. Tránh TTAS với sleep backoff khi có việc ngoài critical section. Khi tranh chấp cao, chỉ dùng MCS khi số thread không vượt số P-core. Trên máy chạy trực tiếp, dùng barrier spin khi số thread không vượt số core, và barrier có `sched_yield` khi vượt. Trong máy ảo, kể cả khi số thread bằng số CPU ảo, dùng barrier quay rồi nhường CPU (`sr_bo_yield`) hoặc `omp_barrier`. Các barrier chờ chặn chậm hơn `sr` hơn 10 lần ở đó. Dùng fetch-and-add cho counter khi có việc giữa hai lần tăng. CAS có backoff chỉ nhanh hơn khi không có việc đó, và khi đó các thread tăng không đều.

Việc chưa làm gồm phiên bản phân tán bằng MPI-3 RMA (phần 4 của đề), counter chia theo thread (mỗi thread một counter riêng, cộng lại khi đọc), và một máy x86 chạy Linux trực tiếp để tách ảnh hưởng của máy ảo.

References

[1] Thomas E. Anderson. The performance of spin lock alternatives for shared-memory multiprocessors. *IEEE Transactions on Parallel and Distributed Systems*, 1(1):6–16, 1990.

[2] Debra Hensgen, Raphael Finkel, and Udi Manber. Two algorithms for barrier synchronization. *International Journal of Parallel Programming*, 17(1):1–17, 1988.

[3] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2 edition, 2020.

[4] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

[5] John M. Mellor-Crummey and Michael L. Scott. Algorithms for scalable synchronization on shared-memory multiprocessors. *ACM Transactions on Computer Systems*, 9(1):21–65, 1991.

[6] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf.

A Cách build và chạy lại

Mã nguồn cần compiler C++20 (Apple clang, g++ hoặc clang++), `make` và Python 3. Phần vẽ hình cần thêm `matplotlib` và `numpy`. Báo cáo biên dịch bằng XeLaTeX (TeX Live). Chi tiết trong `README.md`.

```
make                # biên dịch các bài kiểm tra và các benchmark vào build/
make test           # chạy mọi bài kiểm tra tính đúng đắn
make tsan           # chạy lại dưới ThreadSanitizer
make mutation       # các biến thể cố ý làm hỏng phải bị bộ kiểm tra bắt
make check          # bốn lệnh trên, theo thứ tự
make bench          # chạy toàn bộ đợt đo trên máy rảnh -> results/<máy>/
make figures        # vẽ lại mọi hình và bảng từ results/ -> report/figures/
make report         # biên dịch report/report.pdf
make zip            # đóng gói bài nộp
make full           # cả quy trình: check, bench, figures, report, zip
```

Trên `apple-m5`, đợt đo có 12 152 lần chạy hoàn thành. Thời lượng 1 giờ 25 phút 45 giây gồm cả 2 phút hiệu chỉnh (643 lần chạy thử, không tính vào số trên). Lệnh đã dùng là `python3 scripts/run_bench.py --relax-yield --host apple-m5 --wait 30`, tương đương `make bench` cộng lượt tùy chọn `make bench-relax`. `make bench-dry` in lưới và thời gian ước tính mà không đo.

Hai thí nghiệm phụ trên `apple-m5` (khoảng trống khi trao lock và đo lại lúc tải thấp) dùng `experiments/handoff_gap.cpp` và `experiments/run_extra.py`. Chương trình `handoff_gap` chỉ cần các header trong `include/`, build bằng `mkdir -p build/experiments && c++ -Iinclude -std=c++20 -O2 -pthread experiments/handoff_gap.cpp -o build/experiments/handoff_gap`. Lượt đo lại dùng `build/bench_lock`, nên cần chạy `make` trước. Lệnh `python3 experiments/run_extra.py apple-m5` ghi `handoff.csv`, `lowload.csv` và `run.log` vào `results/apple-m5/extra/`. Số trong báo cáo tính bằng `scripts/report_numbers/extra_analyze.py`.

Nền tảng gồm macOS 27.0.1, Apple clang 21.0.0, Python 3.14.8 (chạy đo chỉ cần thư viện chuẩn). Apple clang không nhận `-fopenmp` nên không có baseline OpenMP. `matplotlib` và `numpy` cài trong một môi trường ảo (`.venv`).

Trên `LAP1184`, nhóm chạy toàn bộ quy trình bằng `make full PYTHON=.venv/bin/python` trong Ubuntu 20.04 trên WSL2. Đợt đo (`run_bench.py --wait 10`) dài 1 giờ 20 phút 36 giây (14 322 lần chạy hoàn thành). Ubuntu 20.04 cần thêm g++ 11 từ PPA `ubuntu-toolchain-r/test`

(g++ 9 mặc định không có `<barrier>`). Nó cũng cần CMake 3.30 từ bản cài sẵn của Kitware (bản 3.16 của Ubuntu dưới mức 3.20 mà `CMakeLists.txt` cần), `python3-venv`, XeLaTeX và `latexmk`. Python 3.8.10 đủ cho các script.

- Git trên Windows mặc định đổi kết thúc dòng thành CRLF khi checkout, và `sh` trong WSL không chạy được `scripts/machine_info.sh` khi đó. Runner bắt lỗi và ghi một `machine.txt` rỗng. Kho mã có `.gitattributes` giữ LF cho các file `.sh`. `machine.txt` của `LAP1184` được sinh lại bằng đúng lệnh của runner, 3 phút sau khi đợt đo kết thúc. Vì vậy dòng nguồn điện và load average trong đó là của lúc sinh lại.
- ThreadSanitizer báo data race giả trên `omp_lock_t`, vì libgomp không được instrument. Bản TSan của `test_locks` và `test_counter` bỏ qua lock OpenMP và in ra việc bỏ qua, như `test_barrier` làm với `omp_barrier`. Tham số `tsan_uninstrumented` vẫn chạy nó để thấy cảnh báo.
- Không chạy `make bench-relax`, vì lựa chọn `isb`/`yield` chỉ có trên AArch64.

Trên `C4M7Y670NT`, đợt đo (`run_bench.py --wait 10`) dài 1 giờ 53 phút 10 giây (13 244 lần chạy hoàn thành), với macOS 26.7.1, Apple clang 21.0.0 và Python 3.9.6. Cách cài giống `apple-m5`.

B Tên các cài đặt

Bảng 8 cho tên của mọi cài đặt, đúng như trong hình, bảng và log. Lock và barrier thư viện chỉ dùng để so sánh.

Tên	Mô tả	Hệ điều hành
Lock		
`tas`	TAS: lặp `exchange(acquire)`, nhả bằng `store(release)`	mọi máy
`ttas`	TTAS: chờ bằng `load(relaxed)`, rồi mới `exchange`	mọi máy
`mcs`	MCS: hàng đợi FIFO, mỗi thread chờ trên nút riêng	mọi máy
`tas_bo_{sleep,spin,yield}`	TAS, gọi chính sách chờ sau mỗi lần thử hỏng (`sleep_for` nhân đôi; quay ngẫu nhiên; quay 64 lần rồi `yield`)	mọi máy
`ttas_bo_{sleep,spin,yield}`	TTAS, gọi chính sách chờ khi thua `exchange`	mọi máy
`std_mutex`, `pthread_mutex`	`std::mutex`, `pthread_mutex_t`	mọi máy
`os_unfair_lock`	`os_unfair_lock` của macOS	macOS
`pthread_spinlock`	`pthread_spinlock_t`	Linux
`omp_lock`	`omp_lock_t` (cần bản dựng có OpenMP)	Linux
Barrier		
`sr`	sense-reversing barrier, chờ bằng quay	mọi máy
`sr_bo_{sleep,spin,yield}`	`sr`, gọi chính sách chờ như lock giữa hai lần đọc cờ	mọi máy
`sr_block`	`sr`, chờ chặn bằng `std::atomic::wait` và `notify_all`	mọi máy
`std_barrier`	`std::barrier` (C++20)	mọi máy
`condvar`	mutex, condition variable và bộ đếm thế hệ	mọi máy
`pthread_barrier`	`pthread_barrier_t` (macOS không có)	Linux
`omp_barrier`	`#pragma omp barrier` (cần bản dựng có OpenMP)	Linux
Counter		
`faa`, `faa_seqcst`	`fetch_add`, thứ tự `relaxed` hoặc `seq_cst`	mọi máy
`cas`, `cas_seqcst`	vòng CAS không backoff, thứ tự `relaxed` hoặc `seq_cst`	mọi máy
`cas_{sleep,spin,yield}`	vòng CAS, gọi chính sách chờ sau mỗi lần CAS hỏng	mọi máy
`lock_X`	biến `long` trong critical section của lock `X` (mọi lock ở trên)	như lock `X`

Bảng 8: Tên các cài đặt. Hai máy macOS (`apple-m5`, `C4M7Y670NT`) có 12 lock, 7 barrier và 19 counter. `LAP1184` (Linux, có OpenMP) có 13 lock, 9 barrier và 20 counter.

Benchmark và bộ kiểm tra cho các giá trị mà compiler có thể gộp hoặc xoá đi qua `pc::opaque()`: đối số N của mỗi lời gọi `is_prime`, địa chỉ counter ở mỗi vòng của bài kiểm tra counter, và giá trị đọc ra của biến đếm trong bài kiểm tra lock. Hàm này chỉ là một lệnh `asm volatile` rỗng với ràng buộc `+r`. Compiler phải coi giá trị trong thanh ghi đã bị sửa, nên không chứng minh được hai lời gọi giống nhau và không gộp hay xoá chúng. Hàm không sinh lệnh máy nào.

C Kết quả kiểm tra, đợt đo và hình bổ sung

	`apple-m5`	`C4M7Y67ONT`	`LAP1184`
Số thread của stress test	1, 2, 4, 10, 20	1, 2, 4, 12, 24	1, 2, 4, 12, 24
`make test`: lock, barrier, counter đạt	12/12, 7/7, 19/19	12/12, 7/7, 19/19	13/13, 9/9, 20/20
barrier với schedule fuzzing đạt	5/5	5/5	5/5
episode barrier được kiểm	2 198 144	2 198 201	2 085 088
lần tăng counter được kiểm	121 573 312	144 627 911	148 879 205
`make tsan`: lock, barrier, counter đạt	12/12, 7/7, 19/19	12/12, 7/7, 19/19	12/12, 8/8, 19/19
bị bỏ qua	handoff của `std_barrier`	handoff của `std_barrier`	`omp_lock`, `omp_barrier`, `lock_omp_lock`
cảnh báo data race	0	0	0
`make mutation`: biến thể bị bắt	14/14	14/14	14/14

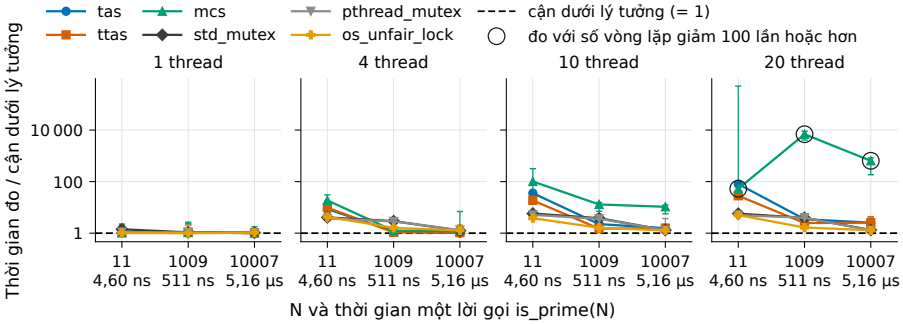
Bảng 9: Kết quả kiểm tra tính đúng đắn trên ba máy, mỗi máy một lượt. Episode barrier chỉ tính bản thường. Lý do bỏ qua trong bản TSan nằm ở các Mục 4.2–4.4.

Bảng 10: Tóm tắt đợt đo trên máy `apple-m5`: số lần chạy, số lần vượt thời hạn và tải nền ghi trong từng dòng số liệu.

	Lock	Barrier	Counter	Lock (`yield`)
Số cài đặt	12	7	19	2
Số cấu hình	1080	70	570	16
Số lần lặp mỗi cấu hình	7	7	7	7
Số lần chạy đã ghi	7 566	490	3 993	112
Cấu hình đo với số vòng lặp giảm	27	0	–	0
Lần chạy vượt thời hạn	6	0	3	0
Lần chạy bỏ qua sau khi vượt thời hạn	0	0	0	0
Lần chạy hỏng phép kiểm (`ok=0`)	0	0	0	0
Load average 1 phút (nhỏ nhất – lớn nhất)	12,4 – 51,2	13,2 – 48,0	12,4 – 51,2	14,2 – 45,2
Bắt đầu (ngày/tháng giờ:phút)	14/10 10:13	14/10 10:13	14/10 10:13	14/10 10:17
Kết thúc	14/10 11:37	14/10 11:36	14/10 11:37	14/10 11:37

Bảng 11: Đơn vị công việc trên máy `apple-m5`: số vòng lặp I của lần chạy 1 thread, thời gian chạy 1 thread của lock nhanh nhất, và t = thời gian đó chia cho số lời gọi `is_prime`, $I\,(1 + \mathrm{OW})$. Nếu với cùng N mà t không đổi theo `OUTSIDE_WORK` thì thời gian tăng tuyến tính theo số lời gọi, tức phần việc không bị compiler xoá. t còn chứa chi phí lấy và nhả lock chia cho $1 + \mathrm{OW}$.

N	`OUTSIDE_WORK`	I (vòng lặp mỗi thread)	Thời gian 1 thread (s)	t (ns)
11	0	43 410 432	0,199	4,60
11	3	10 852 608	0,199	4,58
1009	0	343 663	0,175	511
1009	1	171 832	0,181	526
1009	3	85 916	0,173	504
1009	7	42 958	0,178	519
1009	15	21 479	0,181	527
10007	0	36 381	0,188	5 160
10007	3	9 095	0,189	5 190



Hình 10: Thời gian đo chia cho cận dưới lý tưởng theo độ dài critical section, `OUTSIDE_WORK` = 0, mỗi ô một số thread. Dưới mỗi N là thời gian của một lời gọi `is_prime(N)`.

Bảng 12: Chi phí khi không tranh chấp trên máy `apple-m5`: thời gian chạy 1 thread chia cho số lời gọi `is_prime` (ns), theo N, lấy ở `OUTSIDE_WORK` nhỏ nhất có số liệu. Chênh lệch giữa các hàng là chênh lệch chi phí lấy và nhả lock khi chỉ có một thread.

Lock \ N	11	1009	10007
`tas`	4,69	536	5 210
`tas_bo_sleep`	4,67	516	5 470
`tas_bo_spin`	4,62	549	5 430
`tas_bo_yield`	4,98	518	5 290
`ttas`	4,81	551	5 160
`ttas_bo_sleep`	4,60	559	5 380
`ttas_bo_spin`	4,65	569	5 390
`ttas_bo_yield`	4,60	591	5 210
`mcs`	4,79	538	5 450
`std_mutex`	6,50	548	5 380
`pthread_mutex`	5,10	547	5 430
`os_unfair_lock`	4,71	511	5 400

Bảng 13: Thời gian thật của các phép chờ trên máy `apple-m5`, đo bằng `build/sleep_latency` với một thread: thời gian yêu cầu và thời gian đo được của `sleep_for`, `yield` và lệnh chờ trong vòng spin.

Phép chờ	Yêu cầu (ns)	Số mẫu	Nhỏ nhất (ns)	Trung vị (ns)	Phân vị 99 (ns)	Lớn nhất (ns)
`clock`	0	20 000	0	41,0	42,0	8 330
`sleep_for`	1,00	2 000	3 040	13 200	20 400	145 000
`sleep_for`	10,0	2 000	2 500	4 000	17 700	39 000
`sleep_for`	100	2 000	2 540	2 670	8 170	21 500
`sleep_for`	1 000	2 000	2 540	2 670	7 420	39 600
`sleep_for`	10 000	2 000	13 600	14 700	20 000	27 800
`sleep_for`	100 000	1 000	113 000	128 000	134 000	182 000
`sleep_for`	1 000 000	200	1 060 000	1 260 000	1 280 000	1 280 000
`yield`	0	20 000	125	208	625	92 600
`cpu_relax`	0	2 000	8,50	9,60	21,4	47,8
`isb`	0	2 000	10,2	12,1	21,2	57,9
`yield_insn`	0	2 000	0,200	0,300	0,400	0,400

D Bảng số liệu của hai máy còn lại

Các bảng dưới đây cùng dạng với bảng của `apple-m5` trong mục Phương pháp đo và mục Kết quả và phân tích; mục So sánh giữa các máy trích số từ chúng.

Bảng 14: Tóm tắt đợt đo trên máy `C4M7Y67ONT`: số lần chạy, số lần vượt thời hạn và tải nền ghi trong từng dòng số liệu.

	Lock	Barrier	Counter
Số cài đặt	12	7	19
Số cấu hình	1188	77	627
Số lần lặp mỗi cấu hình	7	7	7
Số lần chạy đã ghi	8 322	539	4 393
Cấu hình đo với số vòng lặp giảm	27	0	–
Lần chạy vượt thời hạn	6	0	4
Lần chạy bỏ qua sau khi vượt thời hạn	0	0	0
Lần chạy hỏng phép kiểm (`ok=0`)	0	0	0
Load average 1 phút (nhỏ nhất – lớn nhất)	10,8 – 48,6	11,6 – 45,0	11,0 – 48,6
Bắt đầu (ngày/tháng giờ:phút)	29/10 19:07	29/10 19:07	29/10 19:07
Kết thúc	29/10 20:58	29/10 20:57	29/10 20:58

Bảng 15: Đơn vị công việc trên máy `C4M7Y67ONT`: số vòng lặp I của lần chạy 1 thread, thời gian chạy 1 thread của lock nhanh nhất, và t = thời gian đó chia cho số lời gọi `is_prime`, $I\,(1 + \mathrm{OW})$. Nếu với cùng N mà t không đổi theo `OUTSIDE_WORK` thì thời gian tăng tuyến tính theo số lời gọi, tức phần việc không bị compiler xoá. t còn chứa chi phí lấy và nhả lock chia cho $1 + \mathrm{OW}$.

N	`OUTSIDE_WORK`	I (vòng lặp mỗi thread)	Thời gian 1 thread (s)	t (ns)
11	0	141 338 304	0,672	4,75
11	3	35 334 576	0,643	4,55
1009	0	312 652	0,156	499
1009	1	156 326	0,156	500
1009	3	78 163	0,156	500
1009	7	39 082	0,156	499
1009	15	19 541	0,156	500
10007	0	36 378	0,180	4 960
10007	3	9 094	0,180	4 960

Bảng 16: Trung vị thời gian chạy (s) của mọi lock trên máy `C4M7Y67ONT`, $N = 1009$, `OUTSIDE_WORK` $= 0$, quy về $I = 312\,652$ vòng lặp mỗi thread (thời gian đo nhân với $I/I_{\text{đo}}$). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy. †: đo với số vòng lặp nhỏ hơn 100 lần (cấu hình sụp đổ hoặc đã vượt thời hạn), rồi quy đổi tuyến tính.

Lock \ số thread	1	2	3	4	6	8	10	12	16	20	24
`tas`	0,156	0,324	0,496	0,687	1,08	1,91	2,53	3,35	5,79	8,39	12,3
`tas_bo_sleep`	0,156	0,326	0,492	0,669	1,03	1,39	1,75	2,10	2,82	3,54	4,29
`tas_bo_spin`	0,156	0,317	0,503	0,682	1,05	2,02	3,64	6,27	10,8	18,8	29,0
`tas_bo_yield`	0,156	0,334	0,502	0,688	1,04	1,45	1,86	2,33	3,81	7,21	5,84
`ttas`	0,156	0,319	0,501	0,676	1,03	1,47	2,04	2,66	4,31	6,34	8,85
`ttas_bo_sleep`	0,157	0,327	0,500	0,666	1,02	1,46	1,95	2,65	4,09	5,00	7,32
`ttas_bo_spin`	0,156	0,324	0,501	0,674	1,03	1,47	1,99	2,67	4,20	6,27	9,39
`ttas_bo_yield`	0,156	0,327	0,501	0,665	1,03	1,47	2,00	2,68	4,73	6,08	8,92
`mcs`	0,156	0,340	0,565	0,754	1,15	3,22	5,41	9,34	5 170†	10 300†	22 200†
`std_mutex`	0,156	0,479	1,24	2,27	2,93	4,50	6,38	7,76	6,33	8,39	9,45
`pthread_mutex`	0,157	0,497	1,30	2,25	2,89	4,51	6,32	7,31	7,42	7,59	10,4
`os_unfair_lock`	0,157	0,445	0,724	0,936	1,47	1,96	2,48	2,98	3,98	5,01	6,04
cận dưới lý tưởng	0,156	0,312	0,468	0,625	0,937	1,25	1,56	1,87	2,50	3,12	3,75

Bảng 17: Trung vị số barrier episode mỗi giây trên máy `C4M7Y67ONT` theo số thread (k = nghìn, M = triệu; t/h = vượt thời hạn).

Barrier \ số thread	1	2	3	4	6	8	10	12	16	20	24
`sr`	530 M	15,2 M	10,7 M	7,29 M	4,66 M	2,03 M	1,58 M	1,35 M	184	104	68,1
`sr_bo_sleep`	531 M	465 k	339 k	207 k	133 k	86,3 k	49,0 k	33,8 k	27,0 k	25,4 k	22,1 k
`sr_bo_spin`	521 M	7,14 M	3,56 M	1,74 M	426 k	203 k	161 k	124 k	173	107	66,1
`sr_bo_yield`	532 M	16,0 M	10,9 M	7,28 M	4,64 M	1,96 M	1,38 M	1,16 M	60,5 k	43,3 k	31,9 k
`sr_block`	210 M	11,3 M	7,96 M	6,11 M	4,13 M	622 k	551 k	55,6 k	36,8 k	30,2 k	25,1 k
`std_barrier`	53,4 M	8,54 M	5,04 M	4,06 M	2,60 M	442 k	323 k	50,9 k	36,0 k	29,8 k	23,2 k
`condvar`	145 M	556 k	315 k	200 k	124 k	66,0 k	48,6 k	39,8 k	29,5 k	25,5 k	20,4 k

Bảng 18: Trung vị thời gian chạy (s) của mọi counter trên máy `C4M7Y67ONT`, `OUTSIDE_WORK` = 0 (lời gọi `is_prime(101)` giữa hai lần tăng), quy về 10^6 lần tăng mỗi thread (thời gian đo nhân với $10^6/I$; I được hiệu chỉnh riêng cho từng cấu hình). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy.

Counter \ số thread	1	2	3	4	6	8	10	12	16	20	24
`faa`	0,00178	0,00696	0,0135	0,0230	0,0484	0,216	0,382	0,259	0,354	0,453	0,684
`faa_seqcst`	0,00173	0,00711	0,0148	0,0238	0,0431	0,250	0,369	0,289	0,442	0,573	0,656
`cas`	0,00192	0,00530	0,0231	0,172	0,539	0,973	1,36	1,78	3,55	2,96	3,63
`cas_sleep`	0,00190	0,00395	0,00607	0,00819	0,0126	0,0169	0,0217	0,0260	0,0352	0,0455	0,0549
`cas_spin`	0,00212	0,00429	0,00655	0,00878	0,0134	0,0189	0,0240	0,0301	0,0430	0,0560	0,0675
`cas_yield`	0,00179	0,00372	0,00598	0,0115	0,0153	0,0216	0,0356	0,0315	0,0420	0,0516	0,0590
`cas_seqcst`	0,00196	0,00577	0,0932	0,206	0,334	0,927	2,02	2,76	2,45	4,96	6,07
`lock_tas`	0,00146	0,0939	0,277	0,450	0,886	1,70	2,57	3,67	5,95	8,09	9,71
`lock_tas_bo_sleep`	0,00141	0,00291	0,00448	0,00626	0,00985	0,0135	0,0174	0,0214	0,0292	0,0418	0,0619
`lock_tas_bo_spin`	0,00156	0,00334	0,00589	0,00904	0,0190	0,0595	0,133	0,286	0,433	0,536	1,13
`lock_tas_bo_yield`	0,00139	0,0178	0,143	0,337	0,525	1,58	2,55	3,31	4,19	5,23	6,25
`lock_ttas`	0,00140	0,112	0,280	0,415	0,845	1,79	2,54	3,24	4,32	5,89	7,14
`lock_ttas_bo_sleep`	0,00147	0,00284	0,00446	0,00603	0,00939	0,0125	0,0158	0,0195	0,0265	0,0338	0,0413
`lock_ttas_bo_spin`	0,00156	0,00319	0,00505	0,00677	0,0108	0,0164	0,0242	0,0432	0,0587	0,0959	0,105
`lock_ttas_bo_yield`	0,00142	0,00633	0,0194	0,0609	0,513	1,06	1,15	2,23	1,41	0,639	1,17
`lock_mcs`	0,00224	0,198	0,249	0,260	0,405	1,11	1,73	2,61	3,25	1 750	3,77
`lock_std_mutex`	0,00450	0,0164	0,273	0,382	0,133	0,143	0,164	0,214	0,294	0,382	0,445
`lock_pthread_mutex`	0,00457	0,0147	0,149	0,153	0,0915	0,121	0,150	0,187	0,241	0,313	0,378
`lock_os_unfair_lock`	0,00185	0,0346	0,342	0,513	0,123	0,110	0,150	0,146	0,210	0,290	0,369

Bảng 19: Tóm tắt đợt đo trên máy `LAP1184`: số lần chạy, số lần vượt thời hạn và tải nền ghi trong từng dòng số liệu.

	Lock	Barrier	Counter
Số cài đặt	13	9	20
Số cấu hình	1287	99	660
Số lần lặp mỗi cấu hình	7	7	7
Số lần chạy đã ghi	9 015	693	4 626
Cấu hình đo với số vòng lặp giảm	27	0	–
Lần chạy vượt thời hạn	6	0	6
Lần chạy bỏ qua sau khi vượt thời hạn	0	0	0
Lần chạy hỏng phép kiểm (`ok=0`)	0	0	0
Load average 1 phút (nhỏ nhất – lớn nhất)	5,90 – 19,6	5,90 – 19,6	5,90 – 19,6
Bắt đầu (ngày/tháng giờ:phút)	29/10 00:23	29/10 00:23	29/10 00:23
Kết thúc	29/10 01:41	29/10 01:41	29/10 01:41

Bảng 20: Đơn vị công việc trên máy `LAP1184`: số vòng lặp I của lần chạy 1 thread, thời gian chạy 1 thread của lock nhanh nhất, và t = thời gian đó chia cho số lời gọi `is_prime`, $I\,(1 + \mathrm{OW})$. Nếu với cùng N mà t không đổi theo `OUTSIDE_WORK` thì thời gian tăng tuyến tính theo số lời gọi, tức phần việc không bị compiler xoá. t còn chứa chi phí lấy và nhả lock chia cho $1 + \mathrm{OW}$.

N	`OUTSIDE_WORK`	I (vòng lặp mỗi thread)	Thời gian 1 thread (s)	t (ns)
11	0	28 364 544	0,368	13,0
11	3	7 091 136	0,361	12,7
1009	0	89 797	0,147	1 640
1009	1	44 898	0,144	1 600
1009	3	22 449	0,144	1 600
1009	7	11 225	0,147	1 640
1009	15	5 612	0,144	1 610
10007	0	8 609	0,140	16 300
10007	3	2 152	0,142	16 500

Bảng 21: Trung vị thời gian chạy (s) của mọi lock trên máy `LAP1184`, $N = 1009$, `OUTSIDE_WORK` $= 0$, quy về $I = 89\,797$ vòng lặp mỗi thread (thời gian đo nhân với $I/I_{\text{đo}}$). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy. †: đo với số vòng lặp nhỏ hơn 100 lần (cấu hình sụp đổ hoặc đã vượt thời hạn), rồi quy đổi tuyến tính.

Lock \ số thread	1	2	3	4	6	8	10	12	16	20	24
`tas`	0,153	0,328	0,470	0,695	1,03	1,51	1,80	2,40	3,63	5,34	7,89
`tas_bo_sleep`	0,158	0,311	0,501	0,625	0,942	1,27	1,69	2,00	2,66	3,48	4,26
`tas_bo_spin`	0,154	0,321	0,491	0,627	1,22	1,65	2,11	2,47	4,43	6,01	8,93
`tas_bo_yield`	0,154	0,335	0,481	0,705	1,14	1,45	1,96	2,37	3,12	4,00	5,26
`ttas`	0,154	0,325	0,467	0,670	1,01	1,30	1,77	2,21	3,24	4,67	6,06
`ttas_bo_sleep`	0,158	0,320	0,493	0,658	1,11	1,45	1,97	2,69	3,68	4,73	7,71
`ttas_bo_spin`	0,154	0,317	0,468	0,635	1,08	1,33	1,76	2,20	3,24	5,10	7,29
`ttas_bo_yield`	0,154	0,309	0,488	0,630	1,07	1,44	1,83	2,19	3,53	4,11	6,15
`mcs`	0,147	0,342	0,497	0,746	1,38	1,97	2,72	5,51	9,66†	2 120†	10,5†
`std_mutex`	0,168	0,575	0,938	1,49	2,39	3,40	4,40	5,33	7,33	8,99	11,1
`pthread_mutex`	0,154	0,578	0,941	1,47	2,42	3,30	4,36	5,55	6,95	9,04	10,9
`pthread_spinlock`	0,152	0,342	0,481	0,652	1,04	1,39	1,78	2,26	3,41	5,14	6,89
`omp_lock`	0,160	0,300	0,491	0,659	1,00	1,37	1,81	2,24	3,26	4,51	6,33
cận dưới lý tưởng	0,147	0,294	0,441	0,588	0,881	1,18	1,47	1,76	2,35	2,94	3,53

Bảng 22: Trung vị số barrier episode mỗi giây trên máy `LAP1184` theo số thread (k = nghìn, M = triệu; t/h = vượt thời hạn).

Barrier \ số thread	1	2	3	4	6	8	10	12	16	20	24
`sr`	128 M	18,1 M	6,84 M	5,18 M	2,88 M	1,82 M	1,35 M	78,2 k	48,0	45,8	46,2
`sr_bo_sleep`	125 M	29,6 k	21,5 k	18,6 k	15,4 k	13,0 k	11,4 k	10,3 k	7,03 k	6,04 k	5,74 k
`sr_bo_spin`	135 M	4,70 M	2,08 M	936 k	102 k	44,3 k	31,8 k	17,2 k	48,5	45,0	46,6
`sr_bo_yield`	126 M	19,7 M	7,46 M	5,07 M	3,06 M	1,88 M	1,37 M	111 k	8,34 k	6,80 k	6,87 k
`sr_block`	67,7 M	1,60 M	25,3 k	14,4 k	8,79 k	7,07 k	5,86 k	5,45 k	4,30 k	3,65 k	3,11 k
`std_barrier`	44,9 M	1,16 M	25,9 k	15,7 k	9,45 k	7,40 k	5,61 k	5,32 k	4,39 k	3,62 k	3,04 k
`condvar`	59,8 M	47,4 k	18,2 k	13,6 k	9,30 k	6,64 k	5,67 k	5,73 k	4,51 k	3,68 k	2,91 k
`pthread_barrier`	9,53 M	80,6 k	21,2 k	13,7 k	9,18 k	6,90 k	5,72 k	5,47 k	4,67 k	3,43 k	3,22 k
`omp_barrier`	9,92 M	6,66 M	5,67 M	4,39 M	2,41 M	1,65 M	1,19 M	1,03 M	4,25 k	3,50 k	3,06 k

Bảng 23: Trung vị thời gian chạy (s) của mọi counter trên máy `LAP1184`, `OUTSIDE_WORK` = 0 (lời gọi `is_prime(101)` giữa hai lần tăng), quy về 10^6 lần tăng mỗi thread (thời gian đo nhân với $10^6/I$; I được hiệu chỉnh riêng cho từng cấu hình). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy.

Counter \ số thread	1	2	3	4	6	8	10	12	16	20	24
`faa`	0,00530	0,0266	0,0461	0,0846	0,156	0,193	0,283	0,367	0,495	0,578	0,756
`faa_seqcst`	0,00508	0,0260	0,0478	0,0775	0,152	0,219	0,273	0,331	0,529	0,635	0,775
`cas`	0,00892	0,0397	0,0931	0,244	0,448	0,759	0,903	1,48	2,03	2,48	3,17
`cas_sleep`	0,00920	0,0192	0,0292	0,0380	0,0561	0,0765	0,101	0,121	0,155	0,213	0,285
`cas_spin`	0,00968	0,0207	0,0298	0,0437	0,0674	0,0977	0,119	0,144	0,183	0,237	0,274
`cas_yield`	0,00895	0,0253	0,0458	0,0901	0,293	0,627	0,951	1,21	2,02	2,37	1,76
`cas_seqcst`	0,00919	0,0403	0,0716	0,184	0,511	0,727	1,03	1,50	1,82	2,33	3,44
`lock_tas`	0,00705	0,0347	0,0953	0,265	0,648	1,27	2,25	1,98	5,37	7,61	9,72
`lock_tas_bo_sleep`	0,00758	0,0155	0,0251	0,0343	0,0591	0,0890	0,114	0,140	0,214	0,301	0,372
`lock_tas_bo_spin`	0,00782	0,0160	0,0256	0,0404	0,0840	0,139	0,188	0,258	0,406	0,598	0,824
`lock_tas_bo_yield`	0,00773	0,0381	0,0998	0,226	0,563	1,29	1,36	3,19	3,82	5,57	7,13
`lock_ttas`	0,00763	0,0552	0,107	0,250	0,667	0,956	1,15	1,53	2,24	2,69	5,84
`lock_ttas_bo_sleep`	0,00764	0,0157	0,0283	0,0354	0,0577	0,0964	0,117	0,182	0,243	1,26	5,82
`lock_ttas_bo_spin`	0,00769	0,0286	0,0619	0,142	0,321	0,544	0,821	1,01	1,82	2,61	3,78
`lock_ttas_bo_yield`	0,00753	0,0308	0,0862	0,336	0,609	0,908	1,17	1,64	3,07	3,08	3,85
`lock_mcs`	0,0170	0,194	0,296	0,410	0,751	1,27	9,18	13,2	21,1	190	10,1
`lock_std_mutex`	0,0153	0,148	0,127	0,198	0,385	0,700	0,954	1,20	1,52	1,81	2,12
`lock_pthread_mutex`	0,0154	0,120	0,136	0,204	0,445	0,764	0,935	1,16	1,48	1,87	2,02
`lock_pthread_spinlock`	0,00924	0,0329	0,131	0,231	0,541	1,05	1,20	1,61	2,41	3,59	5,04
`lock_omp_lock`	0,0151	0,0637	0,139	0,333	0,749	1,09	2,21	2,75	6,47	9,29	11,7

E Cấu trúc mã nguồn

Đường dẫn	**Nội dung**
`include/lock.h`	các lock của phần 1 (TAS, TTAS, biến thể backoff, MCS) và lock thư viện
`include/barrier.hpp`	sense-reversing barrier và các barrier dùng để so sánh
`include/counter.hpp`	các counter của phần 3
`include/backoff.hpp`	các chính sách chờ dùng chung cho cả ba phần
`include/platform.hpp`	cache line, lệnh chờ trong vòng spin, đồng hồ, thông tin máy
`include/workload.hpp`	`is_prime` và hàm `opaque` chống trình tối ưu
`include/bench_common.hpp`	đọc tham số, cổng xuất phát, định dạng dòng CSV
`tests/`	bài kiểm tra tính đúng đắn của ba phần
`bench/`	ba chương trình đo và `sleep_latency`
`mk/`	các mục tiêu `make` của từng phần và của việc đóng gói
`scripts/run_bench.py`	chạy toàn bộ đợt đo, ghi số liệu thô vào `results/`
`scripts/plot.py`	vẽ mọi hình, sinh mọi bảng của báo cáo từ `results/`
`scripts/machine_info.sh`	ghi lại cấu hình máy
`scripts/fake_bench.py`	chương trình giả để thử công cụ, không tạo ra số đo
`results/<máy>/`	số liệu thô của từng máy (CSV), cấu hình máy, nhật ký đợt đo
`report/`	mã nguồn báo cáo, hình và bảng sinh tự động

Bảng 24: Cấu trúc thư mục của bài nộp.

F Khai báo sử dụng công cụ AI

[Cần viết: khai báo việc sử dụng công cụ AI theo quy định của môn học (do nhóm tự viết).]